

AVBGraph: Accumulated Version Blocks for Ingestion and Analytics in Transactional Graph Storage

Sitan Ma
Peking University
Beijing, China
masitan@pku.edu.cn

Lei Zou
Peking University
Beijing, China
zoulei@pku.edu.cn

Yinnian Lin
Peking University
Beijing, China
linyinnian@pku.edu.cn

ABSTRACT

Transactional graph storage must sustain high-rate updates while serving traversal-heavy analytics over consistent snapshots. Existing designs usually favor write throughput through log-structured and versioned updates that hurt scan locality, or scan efficiency through tightly organized adjacency layouts that are costly under concurrent updates. Conventional MVCC further amplifies read overhead when long-running analytics pin multiple snapshots, forcing scans to inspect many invisible versions. We present **AVBGraph**, an in-memory graph storage engine built around AVB (Accumulated Version Blocks). AVBGraph accumulates historical updates into contiguous Version Blocks organized by invalidation epochs, **reducing fine-grained version-chain traversal through block-level pruning and sequential scans**. AVBGraph combines a write-friendly update frontier with a scan-friendly historical layout, and **uses epochs for batched asynchronous Version Block publication and composite timestamps for transaction-level visibility within an epoch**. We implement AVBGraph and evaluate it on graphs up to 1B edges under transactional and analytical workloads. AVBGraph achieves up to **10.6** million edge updates per second (**2.08×** over GTX), while **achieving a geometric-mean analytical slowdown of about 1.2×** relative to static CSR and substantially outperforming prior dynamic graph stores under concurrent multi-task workloads.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Mustingu/AVBGraph>.

1 INTRODUCTION

Graph data management is increasingly important in modern analytics. Many applications in fields such as finance [10, 28, 34], social media [23, 49, 54], and cybersecurity [15] operate on rapidly evolving graphs. These applications require continuous updates together with complex queries [7, 48]. Representative examples include real-time recommendation pipelines like Twitter’s Graph-Jet [23, 49] and financial monitoring that requires atomic multi-edge updates [62, 71]. Modern platforms also support multi-tenancy, where concurrent users run heterogeneous analytics over the same evolving graph [24, 44], from community detection [16] to shortest paths [8]. This pressure is further amplified by temporal learning workloads such as dynamic GNNs, which often access multiple historical snapshots to capture graph evolution [36, 46, 61].

We refer to such systems as Transactional Graph Storage (TGS). A TGS engine must provide high-throughput ingestion, efficient analytical scans, and concurrency [4, 70]. Existing designs typically optimize part of this space. Write-oriented systems append

updates to logs, deltas, or versioned records [71, 72], which reduces update cost but weakens adjacency locality. Analytical scans then suffer from disordered access, indirection, and poor cache locality compared with compact layouts such as CSR [47]. In contrast, scan-oriented systems maintain read-friendly structures such as sorted neighbor lists or compact dynamic indexes [12, 22], but incur structural maintenance and synchronization overhead under concurrent updates [55].

Multi-version concurrency further complicates this trade-off. MVCC allows long-running analytics to observe consistent snapshots while updates continue concurrently [41]. However, TGS workloads often involve heterogeneous analytical tasks. These tasks may start at different times and pin different snapshots, forcing the system to retain multiple historical states simultaneously. In conventional versioned layouts, scans must repeatedly resolve visibility by inspecting obsolete or invisible records, increasing pointer chasing, branch mispredictions, and cache misses [55, 66]. This overhead directly affects the target setting of this paper: concurrent traversal-heavy analytics over an actively updated graph.

Our Approach: AVBGraph. We propose AVBGraph, an in-memory graph storage engine built upon AVB (Accumulated Version Blocks). **The current implementation is memory-resident. Persistence would require additional logging, recovery, buffer-management, and I/O-aware publication mechanisms, which are beyond the storage organization studied here and are left for future work.**

AVBGraph organizes historical versions from the outgoing edges of each source vertex into contiguous blocks. Instead of maintaining history only through independent per-edge version chains, AVBGraph groups records that share an invalidation epoch into a *Version Block (VB)*. This organization allows snapshot analytics to prune irrelevant VBs and sequentially scan candidate records, **reducing fine-grained version resolution and improving locality.**

AVBGraph separates the granularity of historical-version accumulation from the granularity of transaction visibility. Recent updates are first absorbed by a write-optimized frontier and later published into Version Blocks in epoch batches, reducing write-path coordination and disruption to long-running readers. **To avoid tying visibility to epoch boundaries, AVBGraph uses a dual-grained timestamp mechanism: epoch identifiers preserve batched Version Block construction and publication, while intra-epoch commit order provides transaction-level snapshot visibility.** Short-lived locks protect physical state, whereas timestamps determine logical visibility.

Contributions. We make the following contributions:

- We propose *AVBGraph*, a transactional graph storage engine that groups historical versions from the outgoing edges of each vertex

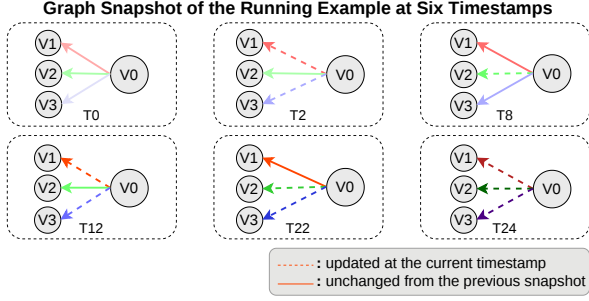


Figure 1: Running example across six timestamps. Darker shades indicate more cumulative modifications; dashed arrows indicate updates at the current timestamp.

into contiguous Version Blocks, improving locality for snapshot traversals.

- We design a *dual-grained timestamp mechanism* that preserves epoch-level batching for efficient historical-version organization while providing transaction-level visibility through intra-epoch commit ordering.
- We implement AVBGraph and conduct experiments on large graphs (up to 1B edges), demonstrating up to 10.6 million edge updates per second (MEPS) (2.08× over GTX) with a geometric-mean analytical slowdown of about 1.2× relative to CSR, outperforming prior dynamic graph systems under concurrent multi-task workloads.

The remainder of this paper is organized as follows. Section 2 reviews the preliminaries. Section 3, 4, and 5 detail the storage and concurrency control of AVBGraph. Section 6 presents the cost model. Section 7 presents the experimental evaluation. Section 8 and Section 9 discuss related work and conclude. The full version [1] further provides appendices with supplementary technical details.

2 PRELIMINARIES AND MOTIVATION

2.1 Graph Model and Adjacency Representations

We model a directed, evolving graph as $G = (V, E)$, where $e = (u, v) \in E$ is an edge from u to v ; undirected graphs are represented by storing both directions. For vertex u , we denote its outgoing neighbor set by $N(u)$ and its out-degree by $d(u) = |N(u)|$.

The graph evolves through snapshots $\{G_0, \dots, G_t\}$ where each snapshot is derived from the previous one by applying an update batch Δ_t , i.e., $G_t = G_{t-1} \oplus \Delta_t$. Updates may insert or delete vertices and edges, or modify a fixed-width edge value. AVBGraph retains snapshot-visible history for these operations. Since traversal is adjacency-oriented, we focus on historical edge versions; an edge value may reference an externally managed property record.

To make the discussion concrete, Figure 1 provides a six-timestamp running example used throughout the rest of the paper: a source vertex v_0 with three edges $e_1 = v_0v_1$, $e_2 = v_0v_2$, and $e_3 = v_0v_3$.

Graph analytics primarily access outgoing adjacency in two ways [22]: (i) *unordered scans* over all neighbors (e.g., BFS [5] and

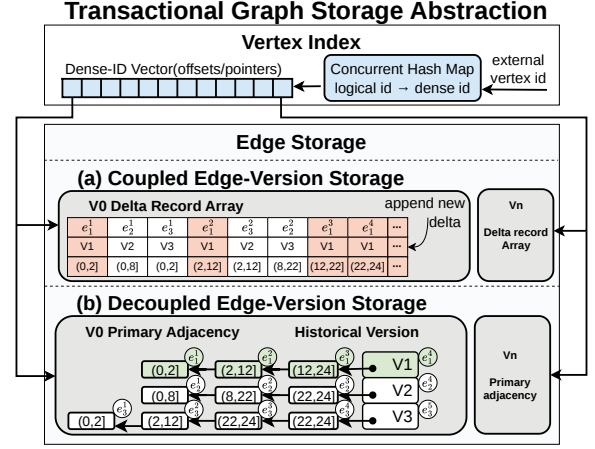


Figure 2: TGS abstraction.

PageRank [42]), and (ii) *ordered access*, typically by destination ID (e.g., triangle counting [60]).

Static layouts such as Compressed Sparse Row (CSR) [47] store edges contiguously with per-vertex offsets, providing excellent scan locality but costly updates due to data movement. Dynamic systems use per-vertex adjacency-list variants to localize inserts and deletes, at the cost of scan fragmentation. Under MVCC (Section 2.2), scans must additionally resolve version visibility, adding a temporal dimension to adjacency traversal.

2.2 Concurrency Control and MVCC

To support concurrent updates and long-running analytics, TGS commonly provides Snapshot Isolation (SI) via Multi-Version Concurrency Control (MVCC). Under MVCC, an update preserves the preceding state as a historical version, allowing readers to traverse a stable snapshot while writers continue updating the graph. In most MVCC-based graph stores, the versioned unit is an edge, and each logical edge version is represented by a version record.

Definition 1 (Version Record and Visibility). *A version record τ is the physical representation of a logical edge version. It stores the edge payload, including the destination vertex identifier and edge value, together with a validity interval $(c(\tau), \text{inv}(\tau))$. Here, $c(\tau)$ is the creation timestamp of τ , and $\text{inv}(\tau)$ is the time when τ is superseded. A record τ is visible to a snapshot at time r iff $c(\tau) < r \leq \text{inv}(\tau)$. The newest committed record has $\text{inv}(\tau) = +\infty$. When no ambiguity arises, we use “record” as shorthand for version record.*

A value modification ends the old version’s validity when the new one begins. Edge-version intervals do not overlap, so a snapshot sees at most one committed edge version and none while the edge is absent.

EXAMPLE 1. Consider edge e_1 which has three records shown in Figure 2(b): $e_1^1(0, 2]$, $e_1^2(2, 12]$, and $e_1^3(12, 24]$. The latest committed state e_1^4 is valid in $(24, +\infty)$. For a snapshot at $r = 10$, only e_1^2 is visible.

2.3 Transactional Graph Storage Abstraction

Most TGS can be abstracted by two components: a vertex index and edge storage, as shown in Figure 2.

Vertex Index. The vertex index maps sparse logical vertex IDs to dense internal IDs and locates per-vertex adjacency regions. It is commonly implemented with a concurrent hash map and vector, such as Sortedton [22] and GTX [71]; AVBGraph follows the same pattern.

Edge Storage. Edge storage largely determines adjacency-scan locality and update scalability, making it a key optimization target in TGS designs [12, 22, 71, 72]. It consists of two logical parts: *primary adjacency* for the current outgoing edges of a vertex and *historical version* for older MVCC records. Existing systems can be classified as coupled or decoupled depending on whether these two parts share physical containers (Figure 2).

I. Coupled edge-version storage (e.g., LiveGraph [72], GTX [71]): These systems store edge updates as delta records, with current states and history sharing the same physical containers. This layout is update-friendly but does not preserve destination order, and snapshot reads must filter invisible records. LiveGraph may scan the delta array for a single-edge lookup, while GTX reduces this cost with hash-chain indexing. Still, accumulated history remains mixed with current records, increasing scan and reclamation overhead.

II. Decoupled edge-version storage (e.g., Teseo [12], Sortedton [22], RapidStore [25]): Decoupling keeps the primary adjacency compact and scan-friendly, but snapshot reads must resolve visibility in external history, typically organized as New-to-Old (N2O) chains. Under high churn and long-running analytics, readers may probe multiple snapshot-invisible records before reaching the visible version, incurring wasted pointer chasing. *RapidStore instead uses subgraph-level snapshot views to avoid repeated per-edge visibility checks, at the cost of coarser-grained publication and update coordination.*

System	VI	Layout	Edge Storage	
LiveGraph	Hash	Coupled	TEL	
GTX	Hash+Vector	Coupled	DeltaBlocks	

System	VI	Layout	PA	HV
Teseo	ART	Decoupled	FatTree+PMA	Edge-Chain
Sortedton	Hash+Vector	Decoupled	SkipList	Edge-Chain
RapidStore	Sg-Vector	Decoupled	C-ART	Sg-Chain
AVBGraph	Hash+Vector	Decoupled	Vector	VB-Chain

Table 1: Component choices under the TGS abstraction. Abbrev.: Sg = Subgraph. VI = Vertex Index. PA = Primary Adjacency. HV = Historical Version. VB = Version Block.

Table 1 summarizes representative systems under this abstraction. This paper focuses on decoupled designs, where external history exposes the cost of multi-version visibility resolution. We analyze this N2O probing cost in Section 2.4.

2.4 Motivating Example: The Cost of N2O Visibility Probing

Figure 3(a) illustrates chain-based visibility resolution on the traversal path. In an N2O layout, each adjacency entry stores the latest committed record and a pointer to older versions. For a snapshot time r , a lookup follows the chain from newer to older records until it finds the first version visible to r .

EXAMPLE 2. Consider snapshot $r=10$ in Figure 3(a). For edge e_1 , the latest record $(24, \infty)$ is newer than r , so the reader rejects $(12, 24]$, and returns $(2, 12]$. Similarly, e_2 rejects $(22, 24]$ before returning $(8, 22]$, while e_3 rejects $(22, 24]$ and $(12, 22]$ before returning $(2, 12]$.

Even in this three-edge example, four historical probes produce no output. Each rejected probe reads version metadata, follows a chain pointer, and may incur cache misses due to fragmented placement. This cost accumulates across many edges and becomes more pronounced under continuous updates, when long-running readers lag behind recent commits and therefore encounter more recent, snapshot-invisible records near the heads of version chains.

This motivates the design goal of AVBGraph: to avoid fine-grained chain probing on the hot traversal path. As shown in Figure 3(b), AVBGraph organizes historical versions into contiguous Version Blocks (Definition 2), enabling block-level pruning and sequential in-block scans instead of repeated per-edge pointer chasing.

3 AVBGRAPH: STORAGE STRUCTURE

3.1 Overview

AVBGraph is a decoupled MVCC storage for transactional graph analytics. For each vertex, the newest committed version of each edge remains in a compact primary adjacency, while older edge versions are reorganized into Version Blocks (Definition 2). Unlike conventional decoupled designs that *use fine-grained per-edge N2O chains as the primary physical organization of history*, AVBGraph groups historical records into contiguous blocks. *This makes block-level pruning and sequential scans over contiguous memory regions the primary history-access path*, while still preserving per-edge version lineage when needed.

At a high level, AVBGraph supports vertex and edge insertions and deletions, as well as fixed-width edge-value modifications. AVB specifically organizes historical edge versions, while vertex insertions and deletions are handled through the vertex index. For analytical reads, a reader accesses the neighborhood of a vertex. A transaction may atomically update multiple edges.

This section first describes AVBGraph’s storage layout and per-vertex organization, then presents snapshot access over this layout, and finally describes the primary adjacency that connects the newest committed state to historical versions.

3.2 AVBGraph Layout

We first define the Version Block as the basic unit of historical storage, and then describe its per-vertex organization and predecessor references for preserving edge-local version lineage.

3.2.1 Version Block. AVBGraph packs historical *version records* of the same source vertex and invalidation timestamp into a contiguous *Version Block*, ordered by creation timestamp.

Definition 2 (Version Block). For a source vertex u and an invalidation timestamp t , a version block $VB_t(u) = \langle \tau_1, \dots, \tau_k \rangle$ is a contiguous sequence of historical version records of u ’s outgoing edges. All records in the block are invalidated at t , i.e., $\text{inv}(\tau_i) = t$ for every i , and are ordered by creation timestamp, i.e., $c(\tau_1) \leq c(\tau_2) \leq \dots \leq c(\tau_k)$. The position of τ_i in the block is denoted by $\text{pos}(\tau_i) = i$.

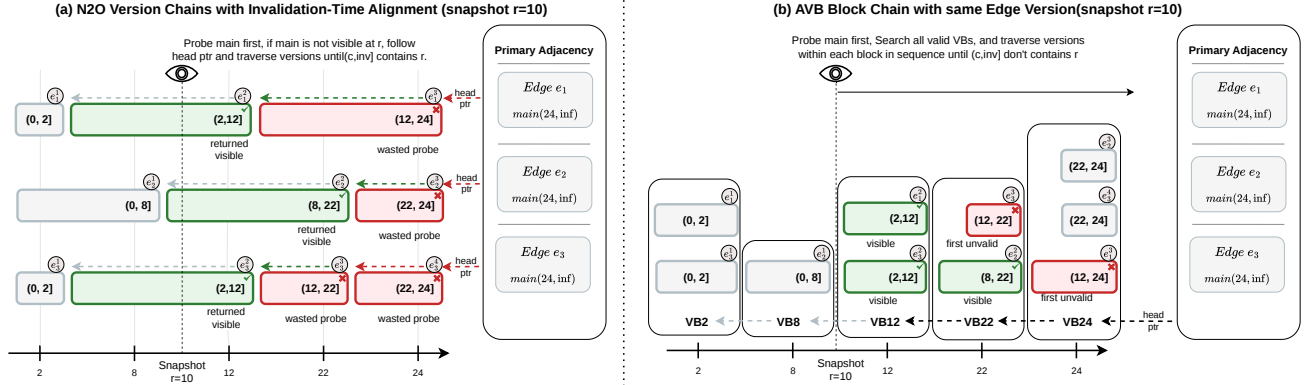


Figure 3: MVCC history access at snapshot $r=10$. Red, green, and gray denote records probed but rejected, returned visible versions, and unvisited older records, respectively. (a) N2O baseline. (b) AVB block layout.

Compared with a conventional N2O layout (Figure 3(a)), AVB-Graph changes the physical organization from edge-centric dispersed chains to time-grouped contiguous blocks.

3.2.2 Per-Vertex Block Organization and Predecessor References. For each vertex, AVBGraph organizes its *version blocks* as a per-vertex block list ordered by decreasing block timestamp (e.g., Figure 3(b)). Hence, versions are ordered by invalidation time, with more recent ones appearing earlier in the list, while older versions are placed in blocks with smaller timestamps.

In MVCC-based access, each edge may require accessing multiple historical versions. In traditional edge-centric dispersed chains (e.g., Figure 3(a)), this can be done easily by following the N2O chain. However, VB changes the physical placement of versions. To preserve the logical predecessor relationship among version records, VB maintains a predecessor reference for each version within blocks when an earlier version of the same edge exists. Specifically, consider such a version τ . Since τ is invalidated at time $inv(\tau)$, it is stored in the block $VB_{inv(\tau)}$. Its immediate predecessor, denoted by $prev(\tau)$, is the version invalidated at the time when τ was created. Therefore, the predecessor of τ resides in the block whose timestamp equals $c(\tau)$.

We represent this relationship using a predecessor reference: $p(\tau) = (t_{prev(\tau)}, pos_{prev(\tau)})$, where $t_{prev(\tau)} = c(\tau)$ denotes the timestamp of the block containing $prev(\tau)$, and $pos_{prev(\tau)}$ denotes the offset of $prev(\tau)$ within $VB_{t_{prev(\tau)}}$. Equivalently,

$$\tau \in VB_{inv(\tau)} \implies prev(\tau) \in VB_{c(\tau)}.$$

Thus, AVBGraph preserves the per-edge temporal lineage without maintaining an explicit per-edge N2O chain. On the read path, this predecessor reference provides a logical N2O-style fallback for order-sensitive snapshot access: when the output adjacency must remain in destination order, the reader scans the ordered primary adjacency and follows predecessor references for each edge until the visible version is found.

EXAMPLE 3 (PREDECESSOR REFERENCE). In Figure 3(b), $\tau = e_1^3$ is stored in VB_{24} with $c(\tau) = 12$, so its predecessor lies in VB_{12} . We record $p(\tau) = (12, 2)$, where 2 denotes the predecessor's second position in VB_{12} , i.e., e_1^2 .

At this point, the *historical version* of AVBGraph is fully defined: each vertex maintains a temporally ordered list of *version blocks*; each block groups records invalidated at the same timestamp; and each record carries a predecessor reference. The next subsection shows that this layout naturally supports efficient snapshot reads.

3.3 Snapshot Read over AVBGraph

We next describe snapshot visibility over the current state and VBs, then compare the resulting read path with conventional N2O history.

3.3.1 Snapshot resolution over AVBGraph. For a snapshot read of vertex u at timestamp r , the reader first examines the newest committed adjacency state. Any current record whose validity interval contains r can be returned immediately. For the historical part, the reader traverses the per-vertex VB chain in decreasing block timestamp order. Let $VB_{t_b}(u)$ denote a version block whose records share the same invalidation timestamp t_b . The read path follows two rules:

- If $t_b < r$, the entire block is skipped.
- If $t_b \geq r$, the reader scans the block sequentially in non-decreasing order of creation timestamps and stops at the first invisible record.

The second rule follows directly from the VB organization. Since all records in $VB_{t_b}(u)$ share invalidation time t_b , $t_b \geq r$ satisfies the invalidation-side condition for the entire block. Visibility then depends only on creation time. Because records are ordered by creation time, once the first record with $c(\tau) \geq r$ is reached, it and all later records are invisible. Thus, a reader can stop the sequential scan at that point.

EXAMPLE 4 (SNAPSHOT READ OVER AVBGRAPH). Consider a snapshot read at $r = 10$ in Figure 3(b). The reader first checks the primary adjacency; since its records are not visible, it continues into the VB chain. VB_8 and VB_2 are pruned because their timestamps are smaller than r , while VB_{24} , VB_{22} and VB_{12} are candidate blocks.

In VB_{24} , the first record has creation time $12 > 10$, so the scan stops immediately. In VB_{22} , $(8, 22)$ is visible and the scan stops before $(12, 22)$. In VB_{12} , the $(2, 12)$ records are visible and returned.

3.3.2 Comparison with N2O. In conventional edge-centric N2O chains, snapshot resolution is performed independently for each edge. A reader starts from the newest version and follows the chain toward older versions until finding a snapshot-visible version or exhausting the chain, potentially probing several too-new records at non-contiguous memory locations.

AVBGraph instead accesses history at block granularity. It skips VBs whose invalidation timestamps precede the snapshot and sequentially scans candidate records in contiguous blocks. Since each VB co-locates versions of outgoing edges invalidated together, one block scan can resolve multiple edges, reducing scattered probes and improving spatial locality. In the example in Figure 3, AVBGraph incurs two wasted probes, versus four for N2O.

3.4 Primary Adjacency

AVBGraph keeps the newest committed state of each vertex in a lightweight primary adjacency, while historical versions are stored in Version Blocks. The primary adjacency serves as the entry point for snapshot reads: a reader first checks the newest records and then scans the relevant Version Blocks when historical versions are needed.

The primary adjacency consists of a *Sorted Edge Array* (SEA) and a small *Write Buffer* (WB). SEA stores stable neighbors in destination order to support sequential scans, whereas WB absorbs recent inserts and updates in an append-friendly form before being periodically sorted and merged into SEA. Each primary-adjacency record contains the destination vertex, edge value, validity metadata, and a reference to its immediate predecessor.

4 EPOCH AND DUAL-GRAINED TIMESTAMP

AVBGraph groups historical edge versions by source vertex and invalidation time. [Versions made historical by the same transaction share its commit timestamp and form per-vertex VBs.](#) Large update transactions naturally produce larger VBs, amortizing visibility checks, publication, and garbage collection while enabling sequential scans.

For small transactions, such as single-edge updates, each invalidation timestamp covers only a few records, producing smaller VBs with less batching benefit. [To recover batching opportunities, AVBGraph introduces an epoch abstraction that groups small write transactions within a short time window. Epochs define the granularity for batching updates and constructing Version Blocks.](#)

Using epoch-level invalidation timestamps allows more records to share the same timestamp and be grouped into larger VBs. [When epoch-level visibility is too coarse for freshness-sensitive reads, dual-grained timestamps use intra-epoch commit order to refine visibility without changing the epoch-based batching granularity.](#)

4.1 Epoch: Definition and Lifecycle

We now define the epoch abstraction and describe how epoch publication maps to the per-vertex Version Block layout.

Definition 3 (Epoch). *An epoch E_e is a write-admission interval identified by e . Every write transaction admitted while E_e is open is assigned to e and belongs to the same epoch-level [publication](#) unit.*

Coarse-grained readers therefore do not distinguish individual commit times within the epoch; they observe the epoch only after its publication boundary becomes visible.

An epoch is a logical visibility boundary rather than a physical history container. When E_e is published, records made historical by its updates are reorganized into per-vertex VBs. For each affected vertex u , these records are stored in $VB_e(u)$, where e serves as the coarse-grained invalidation timestamp, analogous to timestamp t in Section 3. Thus, one epoch may produce VBs for multiple vertices, but at most one $VB_e(u)$ for each pair (e, u) . [For coarse-grained readers, epochs determine visibility boundaries, while VBs remain the physical units for storing and scanning history.](#)

Before publication, records of an unpublished epoch are kept in per-vertex Transient Version Blocks (TVBs). A TVB is append-only and does not enforce creation-time ordering. Once all transactions in an epoch have completed, its TVB records become eligible for background publication, where they are reorganized into immutable VBs ordered by non-decreasing creation timestamp c .

Each epoch moves through four states:

- *Open*: the epoch accepts incoming write transactions.
- *Sealed*: the epoch no longer accepts new transactions, but assigned transactions may still be running.
- *Committed*: all assigned transactions have completed, but [their TVB records have not yet been reorganized into VBs.](#)
- *Published*: the epoch has been reorganized into per-vertex VBs and linked into the published history.

AVBGraph maintains two global epoch markers that separate write admission from read visibility:

- *gwe*: the unique epoch currently in the *Open* state, used for writer admission;
- *gre*: the latest epoch in the *Published* state, bounding coarse-grained reader visibility.

The following state transitions determine when these markers advance.

Epoch State Transitions.

- (1) *Open* $\rightarrow$ *Sealed*. When [the target epoch size \$N\$ is reached, or a configured sealing timeout expires](#), the epoch is sealed; a new *Open* epoch is created and *gwe* advances.
- (2) *Sealed* $\rightarrow$ *Committed*. A sealed epoch becomes *Committed* after all assigned transactions complete.
- (3) *Committed* $\rightarrow$ *Published*. The background publisher processes committed epochs in increasing order, starting from $gre + 1$. For each committed epoch, it reorganizes TVB records into per-vertex VBs, publishes them, and advances *gre*. Consecutive committed epochs may be published in one pass.

Publication remains ordered: a later epoch cannot become visible before an earlier unpublished epoch, even if it commits first.

EXAMPLE 5 (EPOCH LIFECYCLE). *Figure 4 illustrates the lifecycle with epoch size $N = 2$. After T_{x1} and T_{x2} enter E_1 , E_1 is sealed and *gwe* advances to the *Open* epoch E_2 . Once all transactions complete, E_1 becomes *Committed* and is later published, advancing *gre* to E_1 .*

*Publication remains ordered across later epochs. Even if E_3 commits before E_2 , the publisher cannot skip E_2 ; E_3 becomes visible only after E_2 is published. Finally, E_4 is the current *Open* epoch.*

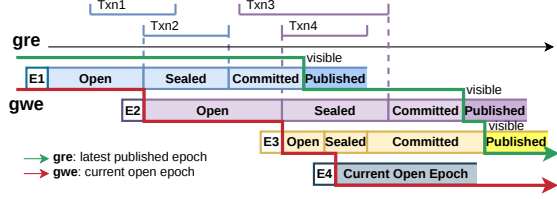


Figure 4: Example of epoch lifecycle in AVBGraph.

4.2 Low-overhead Epoch Management

AVBGraph implements the lifecycle defined in Section 4.1 with two lightweight mechanisms: sampled sealing on the write path and slot-based asynchronous publication. Figure 5 summarizes the workflow.

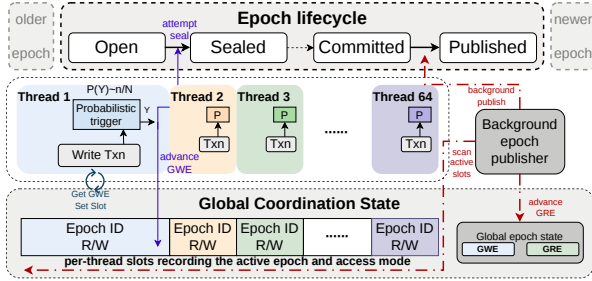


Figure 5: Low-overhead epoch management in AVBGraph.

4.2.1 Decentralized Sampled Sealing. Maintaining an exact per-epoch update counter would make every update touch the same shared counter. AVBGraph instead uses sampled sealing with a small sampling quota h , where $h \ll N$. Each update generates a sealing hit with probability $p = h/N$. A transaction with n updates draws $X_n \sim \text{Binomial}(n, p)$ and adds X_n to the epoch's sampled-hit counter. Crossing h triggers a sealing attempt. Thus, an epoch requires only h expected counter hits rather than N per-update counter increments.

Let U be the number of updates admitted before sealing. Then U follows a negative-binomial distribution with $\mathbb{E}[U] = N$ and $\text{Var}(U) = N(N - h)/h$. Standard Chernoff bounds characterize its concentration; more precisely, for $h = 32$ and large N , the exact negative-binomial distribution gives $\Pr[0.5N \leq U \leq 1.5N] \approx 99.4\%$. At low update rates, a configured timeout triggers sealing before the sampled quota is reached, preventing an epoch from remaining open indefinitely. In practice, we choose the smallest N near write-throughput saturation that meets the epoch-publication delay target. This delay determines freshness in coarse-grained mode, whereas in dual-grained mode N affects batching while transaction visibility follows intra-epoch commit order. Section 7.5 evaluates this trade-off.

When the sampled quota is reached, the triggering transaction acquires a lightweight sealing lock and rechecks whether gwe still points to the same epoch. If so, it seals the epoch, creates a new Open epoch, resets the sampled counter, and advances gwe ; otherwise,

it abandons the attempt. The lock is acquired only during sealing and stays off the common write path.

4.2.2 Asynchronous Commit Detection and Ordered Publication. After an epoch is sealed, AVBGraph detects its completion without a per-epoch active-transaction counter. Each worker records its current epoch and access mode in a local activity slot. The background publisher scans these slots; when the next sealed epoch after gre is no longer referenced by any active writer, it marks the epoch Committed, reorganizes its TVB records into VBs, publishes it, and advances gre . To avoid missing a writer that joins concurrently with sealing, a worker registers through a lightweight guard: it reads gwe , writes that epoch into its slot, and re-reads gwe ; if gwe changed, it retries. Publication proceeds only through consecutive committed epochs, so readers observe a contiguous prefix of published epochs.

Appendix A.3 of the full version [1] provides the detailed sampled-sealing derivation and slot-based publication protocol.

4.3 Dual-Grained Timestamp Extension

AVBGraph uses epochs to batch VB construction. In the coarse-grained mode, a committed transaction becomes visible to readers only when its epoch is published. For freshness-sensitive reads, AVBGraph optionally provides a dual-grained timestamp extension: the epoch remains the unit for batching updates and constructing VBs, while an intra-epoch commit order refines visibility among committed transactions in the same epoch. Prior epoch-based designs may align snapshot boundaries with epochs [59]; others allow committed writes to become visible within an epoch but still delay result release until epoch-level durable commit and replication [35]. In AVBGraph's in-memory setting, transactions can complete before epoch publication; their updates become visible once the snapshot boundary passes their commit timestamps.

Definition 4 (Composite Timestamp). A composite timestamp is a pair $T = (e, t)$, where e is the epoch identifier and t is the intra-epoch commit order. Composite timestamps are compared lexicographically. Under the dual-grained extension, the creation timestamp $c(\tau)$, the invalidation timestamp $inv(\tau)$, and the reader snapshot timestamp T_r are all interpreted as composite timestamps. All updates of a transaction share the same composite commit timestamp, so a snapshot either includes or excludes the transaction across all affected vertices.

Accordingly, a dual-grained $VB_e(u)$ groups records whose invalidation timestamps have the same epoch component e ; their intra-epoch components may differ.

For each vertex u , updates in an unpublished epoch are temporarily recorded in a transient version block $TVB_e(u)$. In the dual-grained write path, a writer holds the write locks of all touched vertices until commit, so uncommitted records are not exposed to readers on the same vertex. The detailed locking protocol is described in Section 5. At commit, the transaction obtains one intra-epoch commit order, assigns the resulting composite timestamp to all its updates, and then releases the locks. TVBs are not sorted or reorganized at commit time; they are reorganized into immutable, creation-ordered VBs only during publication.

A dual-grained reader takes a snapshot timestamp $T_r = (e_r, t_r)$ and examines the primary adjacency, published VBs, and committed

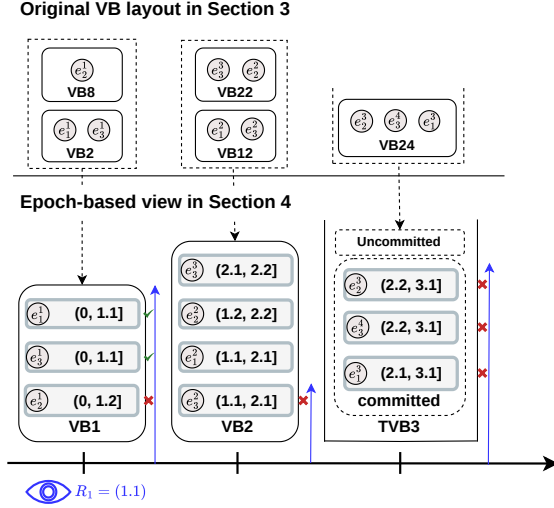


Figure 6: Epoch-based reorganization and dual-grained read paths.

records in relevant unpublished TVBs, which may span multiple epochs. New snapshots do not advance past the earliest epoch with an unfinished writer. A version record τ is visible iff $c(\tau) < T_r \leq inv(\tau)$, where comparisons are lexicographic. We call e_r the reader's active epoch.

Visibility over VBs. Let e_b be the epoch label of a published VB. Dual-grained visibility follows the same block-oriented access path as coarse-grained visibility, except when the block belongs to the reader's active epoch:

- $e_b < e_r$. The block is skipped.
- $e_b > e_r$. The reader scans records in creation-timestamp order and stops once $c(\tau)$ is not earlier than T_r .
- $e_b = e_r$. The reader also scans in creation-timestamp order, but for the scanned prefix it must check the full condition $c(\tau) < T_r \leq inv(\tau)$, because records may be invalidated at different intra-epoch times.

Thus, the only extra work over published VBs appears in the active-epoch case, where epoch-level pruning alone is insufficient to determine visibility.

Visibility over TVBs. A TVB belongs to an unpublished epoch and is not sorted by composite creation timestamp, so it does not support creation-order early stopping. When a reader consults $TVB_e(u)$, it acquires the read lock of vertex u . Because a writer holds the corresponding write lock until commit or abort, the reader cannot observe an in-progress update. The reader then scans the committed records and applies the visibility condition $c(\tau) < T_r \leq inv(\tau)$.

EXAMPLE 6 (READ PATHS UNDER DUAL-GRAINED VISIBILITY). Consider a reader with snapshot timestamp $R_1 = (1, 1)$. For a published VB VB_2 with $e_b > 1$, the reader performs an ordered scan and stops at the first invisible record $(1.1, 2.1]$. VB_1 corresponds to the active-epoch case, so the reader stops at the first record whose creation time is not earlier than R_1 . In this example, it scans all the records. For

the unpublished TVB_3 , the reader scans the committed records and evaluates the full composite-timestamp condition.

Overall, dual-grained timestamps provide transaction-level snapshot visibility while preserving epoch-based grouping, ordered publication, and block-oriented historical layout. Finer checks are needed only for the active-epoch VB case and for records in unpublished TVBs.

5 CONCURRENCY CONTROL

This section describes how readers, writers, and the background publisher coordinate access to shared graph structures. Published VBs are immutable and can be traversed without further synchronization. A per-vertex read-write lock protects the mutable state: the primary adjacency (SEA and WB), TVB, and VB-chain head.

5.1 Concurrency Control Protocol

Shared Execution Workflow. We first introduce the parts of AVBGraph's execution workflow that are shared by both coarse-grained and dual-grained visibility. In both modes, AVBGraph uses three logical worker roles: reader, writer, and publisher.

Reader. A reader acquires the per-vertex read lock to access the primary adjacency and the VB-chain head. It releases the lock before traversing published VBs. In dual-grained mode, it additionally examines the committed records in relevant unpublished TVBs under the same lock.

Writer. A writer acquires the per-vertex write lock and records the update in the vertex-local mutable state. The two modes differ in when the new value is installed in the primary adjacency and how long the writer retains the locks.

Publisher. A publisher acquires the per-vertex write lock, reorganizes accumulated TVB records into immutable VBs, links them to the VB chain, and releases the lock. Its interaction with the primary adjacency differs between the two modes.

Coarse-grained mode. Since updates become visible only at epoch publication, a writer records the new value in the TVB and can release the lock after each edge update. During publication, the publisher reorganizes the TVB into VBs and swaps the latest records with the primary adjacency so that the primary adjacency reflects the newly published state.

Dual-grained mode. Committed updates become visible when snapshots reach their commit timestamps, without epoch publication. A writer installs the newest value in the primary adjacency, moves the displaced value into the TVB, and holds all affected vertex locks until commit or abort. At commit, one composite timestamp is assigned to all updates before the locks are released. Readers inspect relevant unpublished TVBs under the vertex read lock, while the publisher only reorganizes committed TVB records into VBs.

Deadlock and abort handling. AVBGraph uses bounded no-wait locking: if a transaction fails to acquire a required lock, it releases the locks acquired in the current attempt, backs off, and retries, aborting after a bounded number of failures. Abort recovery depends on when updates reach the primary adjacency. In coarse-grained mode, updates remain in the TVB until publication, so abort only removes or marks their TVB records. In dual-grained mode, the writer still holds the affected locks and rolls back the primary

adjacency using undo information in the TVB before removing or marking those records. A sealed epoch is published only after all assigned transactions have committed or aborted, and the publisher materializes only committed records.

5.2 Correctness, Reclamation, and Extensions

We summarize correctness, reclamation, and optional transactional extensions here; Appendices B–D of the full version [1] provide detailed arguments.

Isolation and correctness. The coarse-grained mode exposes consistent snapshots only at epoch-publication boundaries. The dual-grained mode provides transaction-level snapshot isolation: one transaction-wide composite commit timestamp determines visibility across all affected vertices. Per-vertex locks protect vertex-local physical state, write locks order conflicting updates, and readers select committed versions using their snapshot timestamps.

Garbage collection. Obsolete published VBs are reclaimed only after the reader-snapshot frontier has passed them and pre-existing readers have quiesced. Because history is organized into VBs, reclamation proceeds at block granularity rather than along per-edge version chains.

Read-write and serializable extension. The current implementation targets write transactions and read-only snapshot analytics. Serializable read-write transactions can retain read locks until commit, but this extension is better suited to short OLTP-style transactions than to long graph analytics with large read sets.

6 COST MODEL OF N2O CHAINS AND AVB

We develop a quantitative cost model to explain how AVB, the historical-version layout used in AVBGraph, reduces snapshot visibility overhead relative to N2O version chains. Rather than predicting exact runtime, the model captures the dominant memory-access costs of the two layouts. Consider a snapshot scan of vertex v with out-degree d . Let Δ be the lag in epochs from the reader snapshot to the latest state, μ be the mean number of updates per edge per epoch, and c_s and c_r be the costs of sequential and random or pointer-chasing access, respectively. The adjacency of v therefore receives updates at aggregate rate $d\mu$ per epoch.

An edge receives Poisson($\mu\Delta$) updates during the lag window; the probability of at least one update is $1 - e^{-\mu\Delta}$. At the block level, vertex v contributes a non-empty VB in an epoch with probability $1 - e^{-d\mu}$. Let d_h denote the expected number of affected edges and $B_v(\Delta)$ the expected number of non-empty candidate VBs:

$$d_h = d(1 - e^{-\mu\Delta}), \quad B_v(\Delta) = \Delta(1 - e^{-d\mu}).$$

N2O Chain Cost. In an N2O layout, visibility is resolved independently per edge. After scanning the current adjacency, the reader follows version-chain pointers past versions newer than its snapshot. The expected cost is

$$C_{\text{chain}}(v, \Delta) \approx dc_s + d\mu\Delta c_r.$$

Here, dc_s is the scan cost of the current adjacency, while $d\mu\Delta c_r$ captures the dominant overhead of pointer-chasing probes.

AVB Cost. In AVB, historical versions are grouped into VBs. For each VB, a reader pays one random block-entry cost and then scans

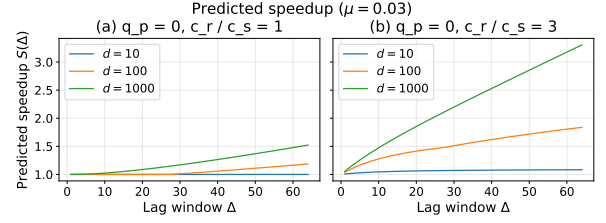


Figure 7: Predicted speedup of AVB over chain-based visibility resolution under two parameter settings.

records sequentially. Let $q_p \in [0, 1]$ be the fraction of candidate VBs that are *pure*, i.e., all records in the VB are visible to the snapshot. For a pure VB, the block-entry access already accounts for the first visible record, so charging all visible records again would overcount one access. For a non-pure VB, the scan may also touch one invisible boundary version, which offsets this overcount. Thus, only pure VBs reduce the effective sequential work:

$$x_{\text{seq}}(\Delta) = \min\{d_h - \min(q_p B_v(\Delta), d_h), d\mu\Delta - B_v(\Delta)\}.$$

The AVB cost is therefore

$$C_{\text{AVB}}(v, \Delta) \approx dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s.$$

Here, the three terms capture the current-adjacency scan, random entry into candidate VBs, and the remaining sequential in-block scan, respectively. The expected speedup over N2O is therefore

$$S(\Delta) = \frac{C_{\text{chain}}(v, \Delta)}{C_{\text{AVB}}(v, \Delta)} \approx \frac{dc_s + d\mu\Delta c_r}{dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s}.$$

Figure 7 illustrates the predicted speedup under two settings. The left subplot uses $q_p = 0$ and $c_r/c_s = 1$, giving AVB neither pure-block credit nor a random-access advantage. The predicted speedup remains close to 1, showing that AVB is not predicted to be worse even under this conservative setting. The right subplot keeps $q_p = 0$ and sets $c_r/c_s = 3$. This ratio reflects reported random-access costs of about 2–3 $\times$ sequential-access costs at 32–64 B [6, 63]. Under this setting, the predicted speedup grows with the lag window Δ , with more pronounced growth at larger d . This is because N2O incurs many edge-level random probes, whereas AVB amortizes visibility resolution over candidate VBs and scans the remaining records sequentially.

Remark on dual-grained visibility. With dual-grained timestamps, a snapshot scan may additionally inspect committed records in unpublished TVBs. Let $R_u(v)$ be the number of such records inspected for vertex v ; this adds $O(R_u(v))$ visibility checks without changing the published-history terms above. For workloads with short write transactions of similar duration, prior MVCC work observes that commit timestamps remain close to the oldest active transaction [9], so the unpublished backlog is usually small.

Appendix E of the full version [1] derives d_h , $B_v(\Delta)$, and the refined $x_{\text{seq}}(\Delta)$ term, including the detailed pure-VB credit and the additional accounting bounds.

7 EXPERIMENTS

Experimental Setup. All experiments ran on a dual-socket server with two 2.10-GHz Intel Xeon Gold 5318Y CPUs (48 physical cores, 96 hardware threads), 72 MiB LLC, and 1.2 TiB memory. Unless

otherwise stated, we use 64 worker threads to create high read-write contention while leaving hardware contexts for the OS and background publication. AVBGraph uses $N = 10^6$ as the target expected epoch size and $h = 32$ as the sampling quota. All systems were compiled with GCC 9.4 using $-O3$. The evaluation driver is publicly available [39].

Dataset. We evaluate AVBGraph on the synthetic and real-world dynamic graphs summarized in Table 2. The synthetic datasets include Graph500 (R-MAT) graphs with power-law degree distributions, Uniform random graphs, and the Graphalytics datagen-7_9-fb dataset (Datagen-7.9); the real-world datasets include DOTA-League and Edit-Wiki.

Dataset	Type	$ V $	$ E $
DOTA-League	Real	61K	51M
Datagen-7.9	Synthetic	1.4M	85M
Edit-Wiki	Real	50M	572M
Graph500-24	R-MAT	9M	260M
Graph500-26	R-MAT	33M	1B
Uniform-24	Uniform	9M	260M

Table 2: Datasets used in the evaluation.

Baselines and Isolation. We compare AVBGraph with GTX [71], LiveGraph [72], Teseo [12], Sortedton (SLT) [22], *RapidStore** [25], and use CSR as an idealized static reference for traversal performance. For read-only analytics, CSR is built from the latest snapshot before each timed run, with edges grouped by source vertex in a contiguous adjacency array indexed by vertex offsets. It remains fixed during the run and maintains no historical versions, transactional metadata, or update paths.

The coarse-grained AVBGraph mode exposes snapshots at epoch-publication boundaries, whereas AVBGraph-dual provides transaction-level snapshot isolation. GTX and Teseo provide snapshot isolation. LiveGraph and SLT are evaluated under their native serializable isolation.

*RapidStore** adds synchronization to reader-trace metadata as a correctness fix and retains the ART property-update path while bypassing unstable non-ART copy-on-write paths. The correctness fix introduces synchronization overhead, whereas bypassing the non-ART paths removes update-maintenance work and biases their performance upward. Because these effects act in opposite directions, we report *RapidStore** as a modified reference. Its memory footprint is omitted; implementation details are given in Appendix F.3 of the full version [1].

Workload and Metrics. Our evaluation covers update-only execution, read-only analytics, concurrent updates and analytics, and memory consumption. The analytics include PageRank (PR), breadth-first search (BFS), single-source shortest path (SSSP), weakly connected components (WCC), local clustering coefficient (LCC), and community detection label propagation (CDLP). We report update throughput, analytics runtime, read latency under concurrent updates, and memory consumption.

7.1 Update Performance

Setup and Workloads. We measure edge-update throughput with 64 writer threads. A write-scalability study on Graph500-24 shows that throughput approaches saturation at high writer concurrency,

supporting this setting; results under varying writer concurrency are reported in Appendix G.2 of the full version [1]. The insertion workload performs edge insertions in random order on all datasets in Table 2. On DOTA-League and Graph500-24, we further evaluate ID (1:1 insertions and deletions), IDM (1:1:2 insertions, deletions, and modifications), and M100 (modifications only). Each update mix executes $5|E|$ operations, where $|E|$ is the number of edges in the initial graph. Additional workload-generation details are provided in Appendix G.1 of the full version [1].

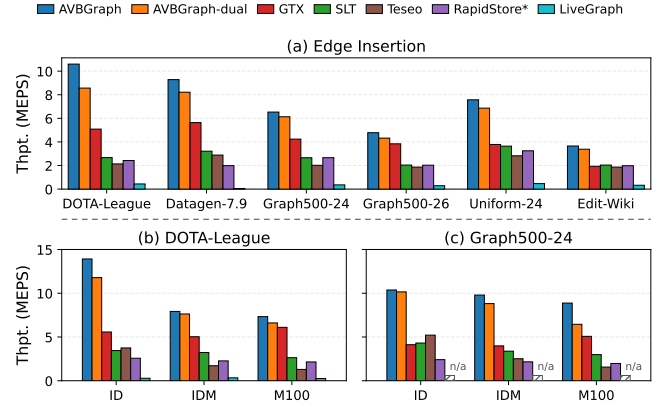


Figure 8: Update throughput with 64 writer threads: (a) edge insertion on all six datasets; (b,c) ID, IDM, and M100 on DOTA-League and Graph500-24. “n/a” denotes runs exceeding four hours.

Insertion throughput. Figure 8(a) shows that AVBGraph achieves the highest insertion throughput on all datasets, reaching 10.60 M inserts/s on DOTA-League and 6.53 M inserts/s on Graph500-24. It outperforms GTX by 2.08 $\times$ and 1.54 $\times$, respectively.

The main reason is that AVBGraph keeps the write path lightweight. New updates are absorbed by the primary adjacency and write-friendly buffers, avoiding immediate reorganization of the compact adjacency layout on each insertion.

AVBGraph-dual has lower throughput than coarse-grained because of additional timestamp and synchronization costs, but still outperforms all baselines across datasets. This shows that transaction-level visibility preserves AVBGraph’s write-side advantage.

General updates. Panels (b) and (c) of Figure 8 show that both AVBGraph modes remain ahead of all baselines with completed runs under ID, IDM, and M100 on both datasets. Against the strongest of these baselines in each configuration, AVBGraph achieves 1.99–2.37 $\times$ higher throughput under ID, 1.58–2.46 $\times$ under IDM, and 1.20 $\times$ and 1.75 $\times$ under M100 on DOTA-League and Graph500-24, respectively. These results show that AVBGraph’s write-side advantage extends beyond insertion to deletion and mixed updates.

7.2 Read-only Graph Analytics

Figure 9 reports read-only analytics on stable snapshots. The workload suite follows LDBC Graphalytics [27] and includes PR, BFS, SSSP, WCC, LCC, and CDLP. We use the same kernel implementation across systems, with system-specific code limited to the

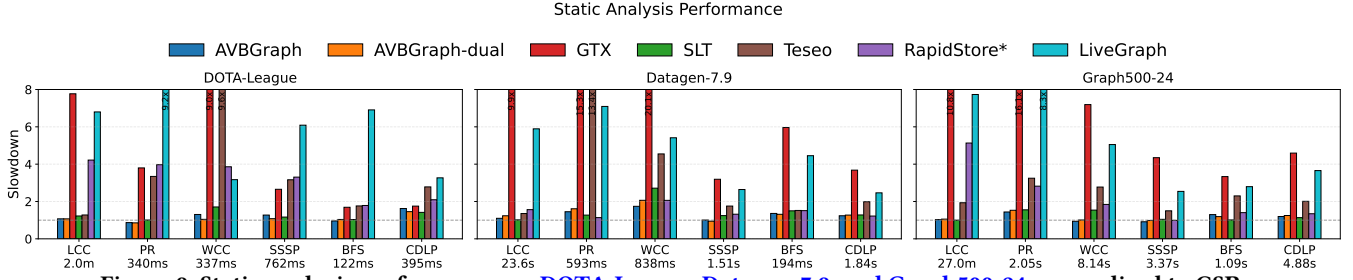


Figure 9: Static analysis performance on DOTA-League, Datagen-7.9, and Graph500-24, normalized to CSR.

adjacency-access layer, and report the mean of three runs normalized to CSR. With no concurrent updates, this experiment isolates current-graph layout and adjacency-scan efficiency rather than snapshot visibility resolution.

Overall performance. Across the evaluated dataset–algorithm pairs, AVBGraph has a geometric mean slowdown of about 1.2× relative to CSR and speedups of 1.7×, 2.2×, 4.8×, and 4.0× over RapidStore*, Teseo, GTX, and LiveGraph, respectively. This shows that AVBGraph does not sacrifice scan efficiency for update support. Its primary adjacency keeps stable neighbors in a compact ordered array and uses a lightweight write buffer for recent updates, so traversal-heavy algorithms can still benefit from mostly sequential adjacency scans.

Analysis. Coupled dynamic stores mix current edges with update or state metadata and may still encounter obsolete entries during scans. This increases scan bandwidth and weakens locality. GTX additionally incurs read-path metadata checks from its lock-free structure. Among the dynamic baselines, SLT generally provides the strongest read-only performance. Both SLT and Teseo maintain ordered dynamic adjacency structures that can benefit order-sensitive workloads such as LCC, but their skip-list and PMA layouts retain additional structural overhead for general analytics. RapidStore* likewise shows no clear advantage for these scan-heavy stable-snapshot workloads.

The read-only results also show little overhead from transaction-level visibility. AVBGraph-dual shares the same current-graph layout as the coarse-grained mode, and on a stable current graph its visibility checks generally succeed without historical-version resolution. Accordingly, the two modes differ by only about 4% in geometric-mean runtime.

7.3 Concurrent Analytics and Updates

Setup and Workloads. We use Graph500-24 with 32 writer threads performing only edge-value updates and 32 reader threads running snapshot analytics. Each run executes approximately $5|E|$ such updates, each in a single transaction. Thus, the main concurrent workload uses 0% insertions, 0% deletions, and 100% value modifications. We consider two update distributions: *Random* samples existing edges uniformly across the graph, following prior concurrent studies [12, 22, 25, 71]. *HotSet*, motivated by the hotspot-update setting in GTX [71], samples the existing outgoing edges of 0.1% of the vertices. Appendix G.3 of the full version [1] further varies the update composition under HotSet with mixed insertions, deletions, and value modifications.

The 32 reader threads are partitioned into 1, 2, 4, or 8 homogeneous PageRank or BFS tasks, or 2, 4, or 8 heterogeneous tasks split equally between PageRank and BFS. We report update throughput and analytical latency. Consistent with prior concurrent evaluations [22, 71], Teseo is excluded due to an unresolved concurrent bug; LiveGraph is omitted because its throughput is at least two orders of magnitude lower in this setting.

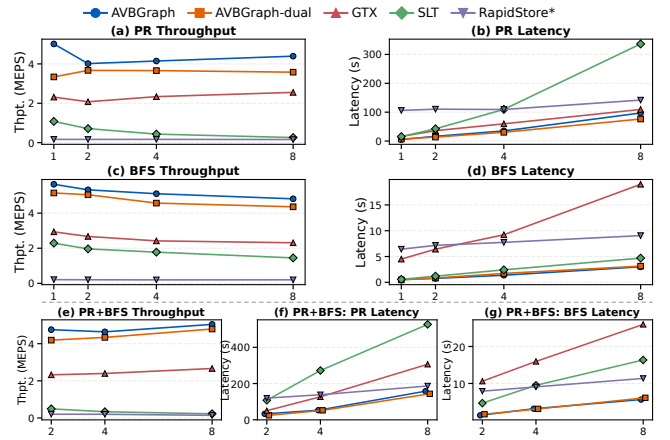


Figure 10: Concurrent update throughput and analytical latency under homogeneous PR and BFS (a–d) and heterogeneous PR+BFS workloads (e–g) on Graph500-24 with Random modifications.

Concurrent Analytical Workloads. Figure 10 evaluates Random modifications with homogeneous PR or BFS readers and heterogeneous PR+BFS readers. PageRank is long-running and scan-intensive, while BFS is a shorter traversal workload, so heterogeneous execution keeps snapshots active for different lengths of time. Across both settings, AVBGraph sustains high update throughput with low analytical latency. As the number of tasks increases, latency generally rises because each task receives fewer reader threads and more snapshots remain active. Since systems sustain different update rates, analytical latency should be interpreted together with the corresponding throughput.

(i) *Homogeneous readers: PR or BFS.* Figure 10(a)–(d) shows the homogeneous workloads. AVBGraph degrades more gracefully as the number of tasks grows because readers can prune irrelevant Version Blocks and sequentially scan applicable records instead of repeatedly resolving history at individual edges. SLT remains competitive for read-only analytics, but its PR latency rises much faster than BFS latency under concurrent updates as long-running PR keeps older snapshots active. GTX continues to pay read-path

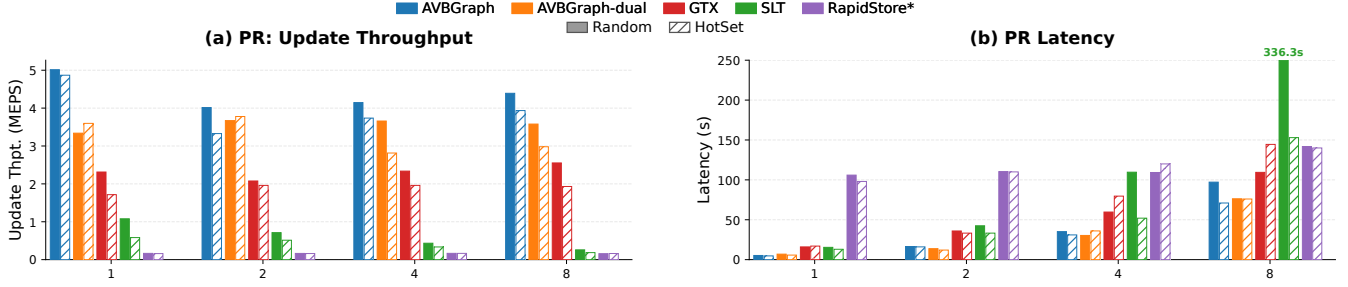


Figure 11: Concurrent update throughput and PageRank latency under Random and HotSet modifications on Graph500-24. The 32 reader threads are partitioned into 1, 2, 4, or 8 PageRank tasks.

metadata and visibility-checking overhead from its coupled version layout. *RapidStore** sustains a substantially lower update rate under concurrent modifications, consistent with its higher coordination and conflict overhead. AVBGraph-dual follows the same overall trend with moderate overhead.

(ii) *Heterogeneous readers: PR+BFS.* Figure 10(e)–(g) shows that AVBGraph continues to sustain high update throughput while keeping both PR and BFS latency low. With snapshots from different times active concurrently, long-running readers keep older versions relevant while other readers operate on newer snapshots. This affects systems that repeatedly resolve visibility at individual edges more strongly. In SLT, BFS latency therefore increases more than in the homogeneous BFS workload when long-running PR tasks keep older snapshots active. This pronounced sensitivity of SLT, a representative per-edge version-chain design, is consistent with AVBGraph’s motivation to organize historical versions across edges into block-contiguous storage. AVBGraph-dual follows the same overall trend while supporting transaction-level visibility.

HotSet Workload. Figure 11 compares HotSet with Random under PageRank. HotSet repeatedly modifies the outgoing edges of a small set of hotspot vertices, concentrating updates and historical versions around these vertices. Because HotSet also changes contention and locality, it stresses concurrent version management under concentrated updates.

AVBGraph is substantially less sensitive to update concentration than GTX and SLT. Across 1 to 8 PR tasks, HotSet reduces the update throughput of the two AVBGraph variants by only 3–20%, compared with 24–42% for GTX and 23–46% for SLT. *RapidStore** changes by less than 4%, but its absolute update throughput remains about 18–30× lower than that of the AVBGraph variants. Notably, lock-free GTX shows a reduction comparable to SLT, indicating that the HotSet effect is not explained by lock contention alone.

Latency should be interpreted together with the update rate. AVBGraph keeps PR latency broadly comparable to Random while maintaining high update throughput. SLT’s latency decreases by 16–55% under HotSet, but this coincides with a 23–46% drop in its update throughput and thus weaker write pressure. GTX instead shows a 24–55% increase in PR latency while its update throughput also drops. *RapidStore** remains relatively stable in latency, but at its substantially lower update rate. These results show AVBGraph’s advantage over *RapidStore** under sustained concurrent modifications, particularly when updates are concentrated on hotspot vertices.

These results are consistent with AVBGraph’s block-oriented history organization. By accumulating displaced versions into Version

Blocks and pruning history at block granularity, both AVBGraph variants remain relatively stable even when modifications and historical versions are concentrated around hotspot vertices.

7.4 Memory Footprint

Experimental setup. We measure process-level RSS externally relative to the initialized state. Because systems finish at different wall-clock times, we normalize execution progress: the first 20% represents insertion and the remaining 80% the concurrent read-write phase. *RapidStore** is omitted because bypassing part of its update-maintenance path makes its memory footprint incomparable.

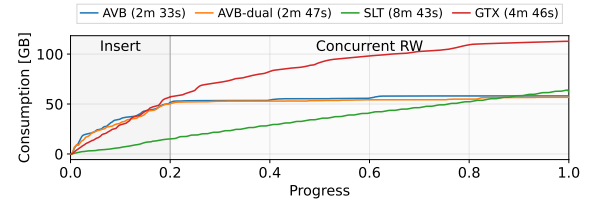


Figure 12: Memory consumption during insertion and concurrent read-write execution.

Results and analysis. Figure 12 shows that both AVBGraph modes quickly flatten in the high-50 GB range after entering the concurrent read-write phase, even while active snapshots retain historical versions. AVBGraph-dual remains close to the coarse-grained mode throughout, with no noticeable additional memory amplification in this experiment.

SLT has a lower insert-phase footprint but takes much longer to finish the same workload: 8m 43s versus 2m 33s for AVBGraph and 2m 47s for AVBGraph-dual. Its memory usage continues to grow during concurrent execution and eventually exceeds both AVBGraph variants. GTX shows the largest growth, ending above 110 GB, consistent with its per-version metadata overhead and delayed reclamation while old snapshots remain active.

7.5 Epoch-size Sensitivity

Figure 13 shows the sensitivity to the target expected epoch size N under 32W+32R on Graph500-24 with PageRank readers. As N increases from 10^3 to 10^7 , AVBGraph’s update throughput increases from 1.78 to 5.49 MEPS and AVBGraph-dual’s from 1.73 to 4.76 MEPS. PageRank latency remains approximately 4–6 s without a monotonic increase.

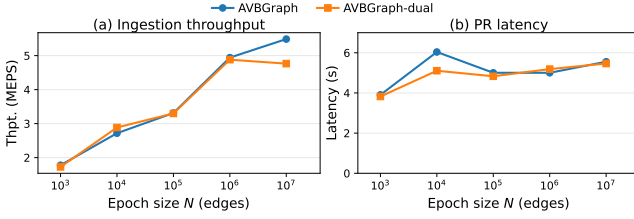


Figure 13: Epoch-size sensitivity under 32W+32R on Graph500-24 with PageRank readers.

Larger epochs produce larger VBs and amortize publication work over more updates, but make reader-visible publication less frequent in the coarse-grained mode. AVBGraph-dual uses the same epoch batching, while transaction visibility is determined by intra-epoch commit order rather than epoch publication. At $N = 10^6$, AVBGraph reaches approximately 4.9 MEPS, within about 11% of its throughput at $N = 10^7$; we therefore use $N = 10^6$ as the smallest tested value near the throughput plateau. In practice, we choose the smallest N near the throughput plateau that satisfies the epoch-publication-delay requirement for the coarse-grained mode; for the dual-grained mode, N can be chosen primarily according to update-throughput saturation.

8 RELATED WORK

Dynamic and temporal graph storage. Static frameworks such as Ligra+ [52, 53] provide CSR-like locality for traversal-heavy analytics but do not support fine-grained online updates. Dynamic graph stores extend compact adjacency with snapshot layers, buffers, or PMA-style gaps [19, 37, 64], or use mutable and adaptive adjacency structures [14, 16, 30, 43]. Other in-memory designs reduce indexing and storage overhead [51, 67], adapt representations to vertex degree [2], improve prefetching [32], or target low-latency incremental analysis [17]. LSMGraph [68] embeds CSR structures in LSM-tree levels for disk-resident dynamic graphs. A systematic study of concurrent in-memory designs identifies memory access, version maintenance, and synchronization as important costs [55].

Beyond the latest dynamic graph, some systems explicitly manage update order or historical states. HAL [3] supports transactional processing of out-of-order graph updates. Temporal graph systems further retain historical states for temporal queries. Aion [56] decouples the latest graph from historical updates using complementary time- and entity-oriented stores; AeonG [26] separates current and historical storage and reconstructs history using anchors and deltas; and TVA [33] separates version metadata from values for efficient temporal neighborhood scans.

General-purpose graph databases. These systems provide broader DBMS functionality, including query languages, indexing, persistence, distributed execution, and transactions. Neo4j [40], TigerGraph [13], and NebulaGraph [65] support operational graph management; ByteGraph [31], BG3 [69], and Galaxybase [58] target large-scale or HTAP graph processing; and GraphflowDB [38] and Kuzu [18] study columnar storage and query execution.

Transactional graph stores. Transactional graph stores extend dynamic graph storage with isolation and atomic updates. Section 2.3 compares representative designs in detail: LiveGraph [72]

and GTX [71] use coupled edge-version storage, whereas Teseo [12], Sortledton [22], and RapidStore [25] use decoupled designs with different adjacency and version organizations.

Among these decoupled systems, RapidStore maintains copy-on-write versions at subgraph granularity. A reader constructs a snapshot view once, avoiding repeated per-edge visibility checks on subsequent accesses. AVBGraph instead retains edge-level updates and groups historical records from a vertex’s outgoing edges into Version Blocks, allowing readers to prune blocks and scan relevant records sequentially.

Long-running read-write graph transactions introduce additional interference beyond these storage trade-offs. Prior work characterizes their behavior [11], schedules short and long-running transactions within epochs [57], or assigns isolation at operation granularity [21]. These works focus on transaction scheduling and isolation rather than the physical organization of accumulated edge history.

MVCC and historical-version organization. Relational MVCC systems organize historical versions of individual records as append-only records, separate history tables, or deltas, linked in old-to-new or new-to-old order [66]. Later designs reduce history-access costs by separating current records from version chains [20], decoupling version indexes from version data [29], or indexing appended versions in a partitioned B⁺-tree [45]. These techniques optimize version access at the individual-record level rather than organizing history across multiple outgoing edges for adjacency-oriented graph access.

Compared with the extensive literature on relational MVCC version representation [20, 45, 66], graph-specific historical-version organization has received less direct attention and is typically embedded in individual storage designs. GART applies coarse-grained MVCC over a mutable CSR representation [50]. A systematic evaluation of in-memory dynamic graph stores further shows substantial space and execution overhead from per-edge version maintenance and visibility checks under concurrent access [55].

AVBGraph retains per-edge visibility semantics but physically groups invalidated records from multiple outgoing edges of the same source vertex by invalidation epoch into contiguous Version Blocks. This cross-edge block organization enables block-level pruning and sequential access to accumulated edge history while retaining edge-local lineage through predecessor references. To the best of our knowledge, AVBGraph is the first transactional graph-storage design to use such a cross-edge block organization as the primary physical organization of historical edge versions.

9 CONCLUSION

AVBGraph, an in-memory transactional graph engine, groups historical edge versions into contiguous Version Blocks by invalidation epoch, reducing fine-grained MVCC chain probes through block pruning and sequential scans. Epoch batching and asynchronous publication sustain updates, while the dual-grained extension supports transaction-level visibility within an epoch. Experiments show high update throughput, analytical performance close to static CSR baselines, and controlled memory growth under mixed read-write workloads.

REFERENCES

- [1] 2026. AVBGraph: Accumulated Version Blocks for Ingestion and Analytics in Transactional Graph Storage (Full Version). [https://github.com/Mustingu/AVBGraph/blob/main/AVBGraph\(fullversion\).pdf](https://github.com/Mustingu/AVBGraph/blob/main/AVBGraph(fullversion).pdf)
- [2] Alif Ahmed, Farzana Ahmed Siddique, and Kevin Skadron. 2024. GraphTango: A Hybrid Representation Format for Efficient Streaming Graph Updates and Analysis. *International Journal of Parallel Programming* 52 (2024), 147–170. <https://doi.org/10.1007/s10766-024-00768-x>
- [3] Angelos Christos Anadiotis, Muhammad Ghufuran Khan, and Ioana Manolescu. 2024. Dynamic Graph Databases with Out-of-order Updates. *Proceedings of the VLDB Endowment* 17, 13 (2024), 4799–4812. <https://doi.org/10.14778/3704965.3704984>
- [4] Renzo Angles, János Benjamin Antal, Alex Averbuch, Altan Birlir, Peter Boncz, Márton Búr, Orri Erling, Andrey Gubichev, Vlad Haprian, Moritz Kaufmann, et al. 2020. The LDBC social network benchmark. *arXiv preprint arXiv:2001.02299* (2020).
- [5] Scott Beamer, Aydın Buluç, Krste Asanović, and David A. Patterson. 2013. *Distributed Memory Breadth-First Search Revisited: Enabling Bottom-Up Search*. Technical Report UCB/EECS-2013-2. EECS Department, University of California, Berkeley. <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2013/EECS-2013-2.html>
- [6] Lawrence Benson, Leon Papke, and Tilmann Rabl. 2022. PerMA-bench: benchmarking persistent memory access. (2022).
- [7] Maciej Besta, Marc Fischer, Vasiliki Kalavri, Michael Kapralov, and Torsten Hoefler. 2021. Practice of streaming processing of dynamic graphs: Concepts, models, and systems. *IEEE Transactions on Parallel and Distributed Systems* 34, 6 (2021), 1860–1876.
- [8] Johannes Blum, Stefan Funke, and Sabine Storandt. 2021. Sublinear search spaces for shortest path planning in grid and road networks. *Journal of Combinatorial Optimization* 42, 2 (2021), 231–257.
- [9] Jan Böttcher, Viktor Leis, Thomas Neumann, and Alfons Kemper. 2019. Scalable Garbage Collection for In-Memory MVCC Systems. *Proceedings of the VLDB Endowment* 13, 2 (2019), 128–141. <https://doi.org/10.14778/3364324.3364328>
- [10] Chandra Chekuri, Kent Quanrud, and Manuel R Torres. 2022. Densest subgraph: Supermodularity, iterative peeling, and flow. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1531–1555.
- [11] Audrey Cheng, Jack Waudby, Natacha Crooks, Hugo Firth, and Ion Stoica. 2023. Mammoths Are Slow: The Overlooked Transactions of Graph Data. *Proceedings of the VLDB Endowment* 17, 4 (2023), 904–911. <https://doi.org/10.14778/3636218.3636241>
- [12] Dean De Leo and Peter Boncz. 2021. Teseo and the analysis of structural dynamic graphs. *Proceedings of the VLDB Endowment* 14, 6 (2021), 1053–1066.
- [13] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor Lee. 2019. Tigergraph: A native MPP graph database. *arXiv preprint arXiv:1901.08248* (2019).
- [14] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. 2019. Low-latency graph streaming using compressed purely-functional trees. In *Proceedings of the 40th ACM SIGPLAN conference on programming language design and implementation*. 918–934.
- [15] Guanghan Duan, Hongwu Lv, Huiqiang Wang, Guangsheng Feng, and Xiaoli Li. 2024. Practical cyber attack detection with continuous temporal graph in dynamic network system. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4851–4864.
- [16] David Ediger, Rob McColl, Jason Riedy, and David A Bader. 2012. Stinger: High performance data structure for streaming graphs. In *2012 IEEE Conference on High Performance Extreme Computing*. IEEE, 1–5.
- [17] Guanyu Feng, Zixuan Ma, Daixuan Li, Shengqi Chen, Xiaowei Zhu, Wentao Han, and Wenguang Chen. 2021. Risgraph: A real-time streaming system for evolving graphs to support sub-millisecond per-update analysis at millions ops/s. In *Proceedings of the 2021 International Conference on Management of Data*. 513–527.
- [18] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoglu. 2023. Kuzu graph database management system. In *The Conference on Innovative Data Systems Research*, Vol. 7. 25–35.
- [19] Soukaina Firmli, Vasileios Trigonakis, Jean-Pierre Lozi, Iraklis Psaroudakis, Alexander Weld, Dalila Chiadmi, Sungpack Hong, and Hassan Chafi. 2020. Csr++: A fast, scalable, update-friendly graph data structure. In *24th International Conference on Principles of Distributed Systems (OPODIS'20)*.
- [20] Michael Freitag, Alfons Kemper, and Thomas Neumann. 2022. Memory-Optimized Multi-Version Concurrency Control for Disk-Based Database Systems. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2797–2810. <https://doi.org/10.14778/3551793.3551832>
- [21] Wenzhi Fu and Yang Cao. 2025. Shifting Transaction Isolation on Graphs: From Systems to Data. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3784–3796. <https://doi.org/10.14778/3749646.3749654>
- [22] Per Fuchs, Domagoj Margan, and Jana Giceva. 2023. Sortedlton: A universal graph data structure. *ACM SIGMOD Record* 52, 1 (2023), 17–25.
- [23] Pankaj Gupta, Venu Satuluri, Ajeet Grewal, Siva Gurumurthy, Volodymyr Zhabuk, Quannan Li, and Jimmy Lin. 2014. Real-Time Twitter Recommendation: Online Motif Detection in Large Dynamic Graphs. *Proc. VLDB Endow.* 7, 13 (2014), 1379–1380.
- [24] Minyang Han and Khuzaima Daudjee. 2015. Giraph unchained: Barrierless asynchronous parallel execution in pregel-like graph processing systems. *Proceedings of the VLDB Endowment* 8, 9 (2015), 950–961.
- [25] Chiyu Hao, Jixian Su, Shixuan Sun, Hao Zhang, Sen Gao, Jianwen Zhao, Chenyi Zhang, Jieru Zhao, Chen Chen, and Minyi Guo. 2025. RapidStore: An Efficient Dynamic Graph Storage System for Concurrent Queries. *Proc. VLDB Endow.* 18, 10 (2025), 3587–3600.
- [26] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. 2024. AeonG: An Efficient Built-in Temporal Support in Graph Databases. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1515–1527. <https://doi.org/10.14778/3648160.3648187>
- [27] Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardt, Hassan Chafi, Mihai Capotă, Narayanan Sundaram, Michael J. Anderson, Ilie Gabriel Tanase, Yinglong Xia, Lifeng Nai, and Peter Boncz. 2016. LDBC Graphalytics: A Benchmark for Large-Scale Graph Analysis on Parallel and Distributed Platforms. *Proceedings of the VLDB Endowment* 9, 13 (2016), 1317–1328. <https://doi.org/10.14778/3007263.3007270>
- [28] Jiaxin Jiang, Yuhang Chen, Bingsheng He, Min Chen, and Jia Chen. 2024. Spade+: A generic real-time fraud detection framework on dynamic graphs. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 7058–7073.
- [29] Jongbin Kim, Jaeseon Yu, Jaechan Ahn, Sooyong Kang, and Hyungsoo Jung. 2022. Diva: Making MVCC Systems HTAP-Friendly. In *Proceedings of the 2022 ACM SIGMOD International Conference on Management of Data*. 49–64. <https://doi.org/10.1145/3514221.3526135>
- [30] Pradeep Kumar and H Howie Huang. 2020. Graphone: A data store for real-time analytics on evolving graphs. *ACM Transactions on Storage (TOS)* 15, 4 (2020), 1–40.
- [31] Changli Li, Hongzhi Chen, Shuai Zhang, Yingqian Hu, Chao Chen, Zhenjie Zhang, Meng Li, Xiangchen Li, Dongqing Han, Xiaohui Chen, et al. 2022. ByteGraph: a high-performance distributed graph database in ByteDance. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3306–3318.
- [32] Hongfu Li, Qian Tao, Song Yu, Shufeng Gong, Yanfeng Zhang, Feng Yao, Wenyuan Yu, Ge Yu, and Jingren Zhou. 2024. GastCoCo: Graph Storage and Coroutine-Based Prefetch Co-Design for Dynamic Graph Processing. *Proceedings of the VLDB Endowment* 17, 13 (2024), 4827–4839. <https://doi.org/10.14778/3704965.3704986>
- [33] Wenhao Li, Zhanhao Zhao, Jinhao Dong, Jiamin Hou, Wei Lu, Yunhai Wang, and Xiaoyong Du. 2026. TVA: A Version-aware Temporal Graph Storage System for Real-time Analytics. *Proceedings of the VLDB Endowment* 19, 10 (2026), 2536–2548. <https://doi.org/10.14778/3828612.3828613>
- [34] Hao Lin, Guannan Liu, Junjie Wu, Yuan Zuo, Xin Wan, and Hong Li. 2019. Fraud detection in dynamic interaction network. *IEEE Transactions on Knowledge and Data Engineering* 32, 10 (2019), 1936–1950.
- [35] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2021. Epoch-based Commit and Replication in Distributed OLTP Databases. *Proceedings of the VLDB Endowment* 14, 5 (2021), 743–756. <https://doi.org/10.14778/3446095.3446098>
- [36] Mingqi Lv, Chengyu Dong, Tieming Chen, Tianian Zhu, Qijie Song, and Yuan Fan. 2021. A heterogeneous graph learning model for cyber-attack detection. *arXiv preprint arXiv:2112.08986* (2021).
- [37] Peter Macko, Virendra J Marathe, Daniel W Margo, and Margo I Seltzer. 2015. Llama: Efficient graph analytics using large multiversioned arrays. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 363–374.
- [38] Amine Mhedhbi. 2023. GraphflowDB: Scalable Query Processing on Graph-Structured Relations. (2023).
- [39] Mustingu. 2026. TransactionGraphSystemDriver. GitHub repository. <https://github.com/Mustingu/TransactionGraphSystemDriver>
- [40] Neo4j, Inc. 2025. *Neo4j Graph Database*. <https://neo4j.com/>
- [41] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast serializable multi-version concurrency control for main-memory database systems. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 677–689.
- [42] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report. Stanford infolab.
- [43] Prashant Pandey, Brian Wheatman, Helen Xu, and Aydın Buluç. 2021. Terrace: A hierarchical graph container for skewed dynamic graphs. In *Proceedings of the 2021 international conference on management of data*. 1372–1385.
- [44] Jiawei Ren, Deji Fu, Chunyu Shi, Zhilan Huang, Wanyi Zhu, and Yi Liu. 2023. Research on Cloud Computing Technology Graph Analysis. In *2023 2nd International Conference on Computing, Communication, Perception and Quantum Technology (CCPQT)*. IEEE, 84–91.
- [45] Christian Riegger and Ilia Petrov. 2024. Storage Management with Multi-Version Partitioned B-Trees. *Information Systems* 125 (2024), 102403. <https://doi.org/10.1016/j.is.2024.102403>

- [46] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020).
- [47] Yousef Saad. 2003. *Iterative methods for sparse linear systems*. SIAM.
- [48] Siddhartha Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M Tamer Özsu. 2020. The ubiquity of large graphs and surprising challenges of graph processing: extended survey. *The VLDB Journal* 29, 2 (2020), 595–618.
- [49] Aneesh Sharma, Jerry Jiang, Praveen Bommannavar, Brian Larson, and Jimmy Lin. 2016. GraphJet: Real-time content recommendations at Twitter. *Proceedings of the VLDB Endowment* 9, 13 (2016), 1281–1292.
- [50] Sijie Shen, Zihang Yao, Lin Shi, Lei Wang, Longbin Lai, Qian Tao, Li Su, Rong Chen, Wenyuan Yu, Haibo Chen, Binyu Zang, and Jingren Zhou. 2023. Bridging the Gap between Relational OLTP and Graph-based OLAP. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 181–196.
- [51] Jifan Shi, Biao Wang, and Yun Xu. 2024. Spruce: a fast yet space-saving structure for dynamic graph storage. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.
- [52] Julian Shun and Guy E Blelloch. 2013. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*. 135–146.
- [53] Julian Shun, Laxman Dhulipala, and Guy E Blelloch. 2015. Smaller and faster: Parallel processing of compressed graphs with Ligra+. In *2015 Data Compression Conference*. IEEE, 403–412.
- [54] Weiping Song, Zhiping Xiao, Yifan Wang, Laurent Charlin, Ming Zhang, and Jian Tang. 2019. Session-based social recommendation via dynamic graph attention networks. In *Proceedings of the Twelfth ACM international conference on web search and data mining*. 555–563.
- [55] Jixian Su, Chiyu Hao, Shixuan Sun, Hao Zhang, Sen Gao, Jiaxin Jiang, Yao Chen, Chenyi Zhang, Bingsheng He, and Minyi Guo. 2025. Revisiting the Design of In-Memory Dynamic Graph Storage. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27. <https://doi.org/10.1145/3709720>
- [56] Georgios Theodorakis, James Clarkson, and Jim Webber. 2024. Aion: Efficient Temporal Graph Data Management. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*. 501–514. <https://doi.org/10.48786/edbt.2024.43>
- [57] Georgios Theodorakis, Hugo Firth, James Clarkson, Natacha Crooks, and Jim Webber. 2025. TuskFlow: An Efficient Graph Database for Long-Running Transactions. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4777–4790. <https://doi.org/10.14778/3750601.3750603>
- [58] Bing Tong, Yan Zhou, Chen Zhang, Jianheng Tang, Jing Tang, Leihong Yang, Qiye Li, Manwu Lin, Zhongxin Bao, Jia Li, and Lei Chen. 2024. Galaxybase: A High Performance Native Distributed Graph Database for HTAP. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3893–3905. <https://doi.org/10.14778/3685800.3685814>
- [59] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy Transactions in Multicore In-Memory Databases. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. 18–32.
- [60] Leyuan Wang, Yangzihao Wang, Carl Yang, and John D Owens. 2016. A comparative study on exact triangle counting algorithms on the gpu. In *Proceedings of the ACM Workshop on High Performance Graph Processing*. 1–8.
- [61] Xuhong Wang, Ding Lyu, Mengjian Li, Yang Xia, Qi Yang, Xinwen Wang, Xinguang Wang, Ping Cui, Yupu Yang, Bowen Sun, et al. 2021. Apan: Asynchronous propagation attention network for real-time temporal graph embedding. In *Proceedings of the 2021 international conference on management of data*. 2628–2638.
- [62] Todd Warszawski and Peter Bailis. 2017. Acidrain: Concurrency-related attacks on database-backed web applications. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 5–20.
- [63] Marcel Weisgut, Daniel Ritter, Pinar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3119–3133.
- [64] Brian Wheatman and Helen Xu. 2021. A parallel packed memory array to store dynamic graphs. In *2021 Proceedings of the Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 31–45.
- [65] Min Wu, Xinglu Yi, Hui Yu, Yu Liu, and Yujue Wang. 2022. Nebula Graph: An open source distributed graph database. *arXiv preprint arXiv:2206.07278* (2022).
- [66] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proceedings of the VLDB Endowment* 10, 7 (2017), 781–792. <https://doi.org/10.14778/3067421.3067427>
- [67] Haoxuan Xie, Junfeng Liu, Siqiang Luo, and Kai Wang. 2026. RadixGraph: A Fast, Space-Optimized Data Structure for Dynamic Graph Storage. *Proceedings of the ACM on Management of Data* 4, 1 (2026). <https://doi.org/10.1145/3786686>
- [68] Song Yu, Shufeng Gong, Qian Tao, Sijie Shen, Yanfeng Zhang, Wenyuan Yu, Pengxi Liu, Zhixian Zhang, Hongfu Li, Xiaojian Luo, Ge Yu, and Jingren Zhou. 2024. LSMGraph: A High-Performance Dynamic Graph Storage System with Multi-Level CSR. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–28. <https://doi.org/10.1145/3698818>
- [69] Wei Zhang, Cheng Chen, Qiang Wang, Wei Wang, Shijiao Yang, Bingyu Zhou, Huiming Zhu, Chao Chen, Yongjun Zhao, Yingqian Hu, Miaomiao Cheng, Meng Li, Hongfei Tan, Mengjin Liu, Hexiang Lin, Shuai Zhang, and Lei Zhang. 2024. BG3: A Cost Effective and I/O Efficient Graph Database in ByteDance. In *Companion of the 2024 International Conference on Management of Data*. 360–372. <https://doi.org/10.1145/3626246.3653373>
- [70] Zhen Zhang, Bingqiao Luo, Shengliang Lu, and Bingsheng He. 2023. Live graph lab: Towards open, dynamic and real transaction graphs with nft. *Advances in Neural Information Processing Systems* 36 (2023), 18769–18793.
- [71] Libin Zhou, Lu Xing, Yeasir Rayhan, and Walid G Aref. 2025. GTX: A Write-Optimized Latch-free Graph Data System with Transactional Support. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.
- [72] Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulmaga, and Wenguang Chen. 2019. LiveGraph: A transactional graph storage system with purely sequential adjacency list scans. *arXiv preprint arXiv:1910.05773* (2019).

A IMPLEMENTATION DETAILS

This appendix provides implementation details that are orthogonal to the main storage design but are useful for understanding how AVBGraph is realized in a dynamic transactional graph store.

A.1 Vertex Index

AVBGraph separates the vertex index from the edge-version storage. The vertex index maps sparse external vertex identifiers to dense internal identifiers, so that per-vertex metadata and adjacency pointers can be stored in compact, array-friendly structures. This design follows the common organization used by transactional graph stores: a concurrent hash map resolves external vertex IDs, while a dense metadata vector stores per-vertex descriptors.

Each vertex descriptor contains the entry points to the vertex-local storage structures, including the primary adjacency, the transient version blocks associated with unpublished epochs, the published VB chain or its head metadata, the vertex-local synchronization object, and lightweight degree or status counters used by traversal and update paths. Since traversal kernels operate on dense vertex IDs after lookup, the dense descriptor array provides predictable locality and avoids repeated hash-table access inside scan-heavy analytics.

To support dynamic growth, the dense metadata vector is implemented as a chunked or two-level vector. New dense IDs are allocated through an atomic ID allocator, and new chunks are created on demand. This avoids relocating existing descriptors when the vertex space grows, so edge-storage pointers and per-vertex metadata references remain stable. *Vertex insertions allocate descriptors through the same path. After a vertex deletion, its versioned state and descriptor are retained while they may still be observed by an active snapshot. Once reclamation is safe, the dense ID is returned to a lock-free free list for reuse.*

The vertex index is not the main optimization target of AVBGraph. Its role is to provide stable per-vertex entry points and to decouple sparse external IDs from the storage layout. For baselines that assume a dense ID domain, we use the same outer sparse-to-dense mapping wrapper when necessary, without modifying their original edge-storage designs. Therefore, the comparison focuses on the edge-version organization rather than on vertex-ID normalization.

A.2 Snapshot Adjacency Traversal Interface

For analytical reads, AVBGraph exposes a common snapshot-aware adjacency traversal interface. The kernels do not directly read the SEA, WB, TVB, or VB representation. Table 3 summarizes the traversal operations used by the evaluated kernels. Each operation evaluates edge visibility using the reader’s fixed snapshot. Consequently, algorithms expressible through vertex-centric adjacency traversal can be added without changing the storage layout. The interface is not a general graph query language and does not provide arbitrary property predicates or query optimization, which require separate execution and property-storage layers. This subsection lists only the read-side traversal operations used by analytics; update operations use separate transaction interfaces and are not enumerated here.

Interface	Operation
TraverseEdges	Visible outgoing edges
...WithProperty	Outgoing edges and edge values
...Until	Traversal with early termination
...Sorted	Neighbors ordered by destination ID

Table 3: Snapshot traversal APIs (... = TraverseEdges).

A.3 Epoch Management Details

This appendix supplements the epoch-management mechanism in Section 4.2 with the derivation behind sampled sealing and the implementation details of asynchronous publication.

Sampled sealing. A direct implementation could maintain an exact per-epoch update counter and seal the epoch once the counter reaches a target size N . However, this would make every update contend on the same shared counter. AVBGraph instead uses sampled sealing. Each epoch maintains a small sampling quota h , where $h \ll N$, and each individual update generates a sealing hit with probability

$$p = \frac{h}{N}.$$

For a transaction containing n edge updates, the number of sampled hits is

$$X_n \sim \text{Binomial}(n, p).$$

The transaction adds X_n to the epoch’s sampled-hit counter, and only when the counter reaches h does a worker attempt to seal the epoch. Thus, the shared counter is touched only by sampled hits rather than by every update.

Let U be the number of individual updates needed to obtain h sampled hits, ignoring the bounded overshoot of the final sealing transaction. Then U follows a negative-binomial distribution:

$$\Pr[U = u] = \binom{u-1}{h-1} p^h (1-p)^{u-h}, \quad u \geq h.$$

Therefore,

$$\mathbb{E}[U] = \frac{h}{p} = N,$$

and

$$\text{Var}(U) = \frac{h(1-p)}{p^2} = \frac{h(1-h/N)}{(h/N)^2} = \frac{N(N-h)}{h}.$$

Hence, sampled sealing preserves the target expected epoch size N , while the sampling quota h controls concentration. In particular, the relative standard deviation is

$$\frac{\sqrt{\text{Var}(U)}}{\mathbb{E}[U]} = \sqrt{\frac{N-h}{hN}} \approx \frac{1}{\sqrt{h}} \quad \text{when } N \gg h.$$

Thus, increasing h makes the realized epoch size more stable, but the expected number of sampled hits per epoch remains only h .

Concentration. The concentration of U can be related to standard Chernoff bounds by viewing sampled hits in a fixed prefix of updates. Let $Y_m \sim \text{Binomial}(m, p)$ be the number of hits among the first m updates. For $0 < \epsilon < 1$, the event $U < (1 - \epsilon)N$ implies that at least h hits occur within $(1 - \epsilon)N$ updates, while the event $U > (1 + \epsilon)N$ implies that fewer than h hits occur within $(1 + \epsilon)N$ updates. Applying Chernoff bounds gives

$$\Pr[U < (1 - \epsilon)N] \leq \exp\left(-\frac{h\epsilon^2}{2 - \epsilon}\right),$$

and

$$\Pr[U > (1 + \epsilon)N] \leq \exp\left(-\frac{h\epsilon^2}{2(1 + \epsilon)}\right).$$

Therefore,

$$\Pr[(1 - \epsilon)N \leq U \leq (1 + \epsilon)N] \geq 1 - \exp\left(-\frac{h\epsilon^2}{2 - \epsilon}\right) - \exp\left(-\frac{h\epsilon^2}{2(1 + \epsilon)}\right).$$

This bound is conservative. For example, when $h = 32$, the exact negative-binomial distribution gives $\Pr[0.5N \leq U \leq 1.5N] \approx 99.4\%$ for large N , showing that a small sampling quota already keeps epoch sizes close to the target while avoiding a per-update global counter.

Concurrent sealing. When a transaction observes that the sampled-hit counter has reached the quota h , it acquires a lightweight sealing lock and rechecks whether gwe still points to the same open epoch. If another worker has already advanced gwe, the attempt is aborted. Otherwise, the worker seals the current epoch, creates a new open epoch, resets the sampled-hit counter, advances gwe, and releases the lock. This lock does not serialize ordinary epoch admission: most transactions produce no sampled hit, and even sampled-hit updates only contend on the sampled counter. The sealing lock is acquired only when the quota is crossed, so it stays off the common write path.

Slot-based commit detection and ordered publication. After an epoch is sealed, AVBGraph must determine when all writers assigned to that epoch have finished. Maintaining a per-epoch active-transaction counter would again introduce centralized write-path synchronization. Instead, each worker records its current epoch and access mode in a local activity slot. The publisher periodically scans these slots to determine whether the next sealed epoch after gre is still referenced by any active writer.

To avoid missing a writer that enters an epoch concurrently with sealing, a worker registers through a lightweight guard: it first reads the current gwe, writes that epoch into its activity slot, and then rereads gwe. If gwe has changed, the worker retries registration. This read-gwe, publish-slot, reread-gwe sequence ensures that the publisher cannot skip a writer that joined an epoch during the scan.

The publisher always processes epochs in increasing order. Once the next epoch after gre is sealed and no active writer slot references it, the publisher marks it committed, transforms its TVB records into per-vertex VBs, links those VBs into the corresponding vertex histories under vertex-local locks, and advances gre only after all affected vertices have been processed. Later epochs cannot bypass an earlier unpublished epoch, even if they become committed earlier. This ordered publication rule preserves a monotone reader-visible

prefix and prevents readers from observing a partially published epoch.

B SNAPSHOT-VISIBILITY CORRECTNESS

This appendix gives the snapshot-visibility argument for the two visibility modes of AVBGraph. The goal is to show that readers observe only committed versions, never observe a partially published epoch, and see all updates of a transaction at one visibility boundary.

B.1 Safety Invariants

We first state the invariants shared by both modes.

Invariant 1: Immutable published VBs. Once a VB is linked into the per-vertex VB chain, it is never modified in place. Later updates may create newer VBs and link them ahead of the existing chain, but the contents of an already published VB remain stable.

Invariant 2: Vertex-local structural consistency. For each vertex, the primary adjacency, TVB state, and VB-chain head are modified only while holding the vertex-local write lock. A reader acquires the read lock to capture the primary-adjacency handle and the VB-chain head, and then traverses immutable published VBs without further synchronization.

Invariant 3: Per-edge lineage preservation. When an update replaces the latest version of an edge, the displaced version is recorded as historical state with the corresponding invalidation timestamp. Thus, for each edge, the primary record and its historical records still form a logical MVCC lineage, even though AVB physically groups historical records by invalidation epoch inside VBs.

Invariant 4: Commit-state consistency. Records from aborted transactions are removed or ignored before publication, and uncommitted records are never exposed as visible historical versions. In coarse-grained mode, the publisher materializes only committed records. In dual-grained mode, readers access unpublished TVB records only under vertex-local protection, and only committed records can be observed.

Invariant 5: Ordered publication. The global read epoch gre advances only after all VBs of the next epoch have been linked into the corresponding vertex histories. Publication is performed in epoch order, so a later epoch cannot become reader-visible before an earlier epoch.

Invariant 6: Atomic transaction boundary. All updates of the same transaction share one reader-visible boundary. In coarse-grained mode, this boundary is the transaction's epoch. In dual-grained mode, it is the transaction's composite commit timestamp. Therefore, a reader cannot observe only a subset of a transaction due to different timestamps on different vertices.

Invariant 7: Fixed reader snapshot. A reader fixes its snapshot boundary at the beginning of the scan and applies the same visibility rule throughout the traversal. Later epoch advancement or publication does not change the snapshot boundary of an already running reader.

B.2 Coarse-Grained Mode

In coarse-grained mode, a reader observes the graph at an epoch boundary. Let e_r be the reader's snapshot epoch. The reader consults

the primary adjacency and published VBs, but does not inspect TVBs from open, sealed, or committed-but-unpublished epochs.

No uncommitted records. A sealed epoch becomes committed only after all writers assigned to that epoch have completed. During publication, aborted records are ignored and only committed updates are transformed into VBs. Since coarse-grained readers do not inspect unpublished TVBs, they cannot observe records from active or aborted transactions.

No partially published epoch. The publisher advances gre only after the entire epoch has been published for all affected vertices. Therefore, if a reader chooses $e_r \leq gre$, every epoch included in its snapshot has already been fully linked into the relevant vertex histories. If an epoch has not been fully linked, gre has not advanced past it, and that epoch cannot be selected as visible by a coarse-grained reader.

Atomic visibility of transactions. All transactions admitted to the same epoch share the same coarse-grained visibility boundary. Hence, for a reader at epoch e_r , an epoch is either included in the published prefix or excluded from it. A transaction cannot be partially visible across vertices because publication does not expose the epoch until all per-vertex VBs of that epoch have been linked.

Snapshot consistency. For each edge, AVB preserves the same logical version intervals as conventional MVCC: a version record τ is visible to snapshot e_r iff $c(\tau) < e_r \leq inv(\tau)$. AVB changes only the physical placement of historical records, grouping them into VBs by invalidation epoch. Because published VBs are immutable and the reader uses a fixed e_r , the reader observes a consistent epoch-level snapshot.

Therefore, coarse-grained mode provides snapshot isolation at epoch granularity. Transactions in the same epoch intentionally share one reader-visible commit boundary, which trades per-transaction freshness for batched publication and low write-path coordination.

B.3 Dual-Grained Mode

Dual-grained mode refines epoch-level visibility with an intra-epoch commit order. A reader takes a composite snapshot timestamp $T_r = (e_r, t_r)$, and timestamps are compared lexicographically. A version record τ is visible iff

$$c(\tau) < T_r \leq inv(\tau).$$

No uncommitted records. In dual-grained mode, a writer holds the write locks of all touched vertices until the transaction commits or aborts. Before commit, readers cannot acquire the corresponding read locks and therefore cannot observe the transaction’s uncommitted primary-adjacency or TVB state. If the transaction aborts, the writer still holds the affected locks and rolls back the primary adjacency using the undo information in the TVB, then removes or invalidates its TVB records. Thus, aborted or uncommitted records are not visible to readers.

Atomic transaction visibility. At commit time, a transaction obtains a single composite commit timestamp. All records produced by the transaction use this same timestamp boundary. The writer releases the touched vertex locks only after all updates of the transaction have been installed or marked committed. [New snapshots do not advance beyond the earliest epoch with an unfinished writer. A later-epoch commit is therefore included only when the snapshot](#)

[boundary reaches its epoch. A reader whose fixed snapshot is later than a transaction’s commit timestamp observes all its updates under the timestamp rule; a reader with an earlier snapshot observes none.](#)

Correctness over published VBs. For published VBs, dual-grained readers use the same block-oriented access path as coarse-grained readers, with additional intra-epoch checks only when needed. If a block’s epoch is older than the reader’s active epoch, the block is outside the visible interval and can be skipped. If it is newer, the reader can scan in creation-timestamp order and stop once records are no longer earlier than T_r . If the block belongs to the reader’s active epoch, the reader checks the full condition $c(\tau) < T_r \leq inv(\tau)$, because records in the same epoch may differ in intra-epoch commit order. These checks are exactly the MVCC visibility rule over composite timestamps.

Correctness over unpublished TVBs. An unpublished TVB is not yet reorganized into a creation-ordered immutable VB, so it does not provide the same early-stopping property as a published VB. [Several TVBs may coexist across unpublished epochs; the reader applies its fixed snapshot timestamp to each relevant one.](#) When a dual-grained reader needs to inspect such a TVB, it does so while holding the vertex-local read lock. This excludes concurrent writers that may be committing or aborting on the same vertex. The reader then tests each committed TVB record using the full composite timestamp condition. Hence, unsorted TVB layout affects only scan cost, not visibility correctness.

Snapshot consistency across vertices. A reader fixes T_r before traversal and applies the same timestamp rule at every vertex. Although different vertices may be read at different physical times, the logical visibility decision depends only on T_r , the record creation timestamp, and the record invalidation timestamp. Since all updates of one transaction share the same composite timestamp, the reader cannot observe a transaction on one vertex but miss it on another due to inconsistent timestamp boundaries.

Therefore, dual-grained mode provides transaction-level snapshot visibility [across all affected vertices. The transaction-wide composite timestamp defines atomic visibility, while the per-vertex locks protect only the vertex-local physical state.](#) Compared with the coarse-grained mode, it exposes committed updates within the [reader’s active](#) epoch earlier, while preserving AVB’s immutable published VBs, ordered publication, and block-oriented historical layout.

C SAFE RECLAMATION

This appendix describes why obsolete published VBs can be reclaimed safely. The key safety requirement is stronger than visibility alone: a reclaimed record must be neither visible to nor reachable by any active reader. This is important because AVBGraph supports both block-oriented historical scans and, when ordered access is required, predecessor-reference traversal from the primary adjacency.

AVBGraph uses delayed, epoch-based reclamation. Each reader publishes its snapshot boundary before it captures any vertex-local entry point, and clears its reader state after the scan finishes. [New readers start only at the current visible snapshot boundary; the interface does not open a new read at an older boundary.](#) Let $(\mathcal{R})$

denote the set of active readers, and let (T_r) be the snapshot boundary of reader (r) . The reclamation frontier is defined as the oldest active reader boundary:

$$T_{\min} = \min_{r \in \mathcal{R}} T_r.$$

If there is no active reader, $(T_{\min})$ is treated as the latest published boundary. For a published VB (B) , let $(ub(B))$ denote the largest invalidation boundary of records contained in (B) . Since VBs are organized by invalidation epochs, this value is simply the block invalidation epoch in coarse-grained mode, and the corresponding composite upper bound in dual-grained mode.

A published VB is eligible for retirement only if

$$ub(B) < T_{\min}.$$

This condition means that every record in (B) has been invalidated strictly before the oldest active snapshot. Therefore, no active reader can observe any record in (B) as visible under the MVCC rule

$$c(\tau) < T_r \leq inv(\tau).$$

If $(inv(\tau) < T_{\min} \leq T_r)$, then $(T_r \leq inv(\tau))$ is false for every active reader.

Retirement and physical freeing are separated. Once a VB satisfies the frontier condition, AVBGraph unlinks it from the corresponding vertex history and places it into a retire list. A reader registered after the frontier test has a snapshot no earlier than $T_{\min}$ and therefore cannot observe any record in the retired block. Readers that may already hold a reference to the block are protected by the subsequent grace period. The memory of the retired VB is freed only after all readers that were active at the time of retirement have exited. This prevents a reader that has already captured an old VB-chain head from dereferencing freed memory.

This two-step rule also covers ordered scans that use predecessor references. Consider an active reader with snapshot (T_r) . If a historical record (τ) has $(inv(\tau) < T_r)$, then (τ) is already obsolete for that reader. Moreover, following predecessor references beyond such a record cannot lead to a visible version for (T_r) , because older predecessors have even earlier validity intervals. Hence, a block whose upper invalidation boundary is smaller than (T_r) is not needed as a visible target or as a necessary intermediate record in predecessor traversal. If a reader may still hold a raw pointer to that block because it captured the chain before unlinking, the grace period delays physical freeing until that reader finishes.

Therefore, after the frontier condition and the grace period both hold, no active reader can either observe or safely reach the reclaimed VB. Future readers cannot reach it because it has already been unlinked from the published history, and existing readers have quiesced before the memory is released.

The same argument applies independently to each vertex history because published VBs are immutable after linking and reclamation removes only blocks below the reader frontier. The primary adjacency and TVB state remain protected by the vertex-local lock. Transient TVB storage is reclaimed separately as part of the epoch publication lifecycle: unpublished records are protected by vertex-local locks while they may be inspected by dual-grained readers, and only committed records are materialized into immutable VBs. After publication, obsolete published history is reclaimed by the block-oriented rule above.

Compared with traditional per-edge version chains, this policy allows AVBGraph to reclaim history at block granularity. Since many obsolete records with nearby invalidation boundaries are accumulated into the same VB, reclamation can retire and free contiguous historical regions instead of chasing and unlinking individual edge-version nodes.

D OPTIONAL TRANSACTIONAL EXTENSIONS

This appendix discusses two optional extensions of AVBGraph: read-write transactions and serializable execution. These extensions reuse the same vertex-local protection mechanism as the main protocol and are not the focus of our evaluation.

D.1 Read-Write Transactions

Read-write transactions can be supported most naturally in the dual-grained mode. A transaction fixes a snapshot timestamp at start time for ordinary snapshot reads. For vertices that it updates, the transaction retains the corresponding vertex-local write locks until commit or abort. Its newly written records are kept either in its private write set or in TVB records tagged with the transaction state, so later reads in the same transaction first consult this private overlay before falling back to the committed snapshot state. This provides read-own-writes semantics without exposing uncommitted records to other readers.

At commit time, the transaction obtains a single composite commit timestamp, and all of its updates are assigned this same timestamp boundary. The updates then follow the normal dual-grained visibility path: the newest values are installed in the primary adjacency, the displaced historical records remain in the TVB, and the publisher later materializes committed TVB records into immutable VBs. If the transaction aborts, it still holds the touched vertex-local locks and can roll back the primary adjacency using the undo information recorded in the TVB or private write set.

This extension preserves snapshot isolation because public readers still apply the same timestamp visibility rule and never observe uncommitted records. The only additional rule is local to the running transaction: its own reads are evaluated over the committed snapshot plus its private updates.

D.2 Serializable Extension

A serializable variant can be obtained by retaining read locks as well as write locks until transaction end, yielding a strict two-phase-locking style protocol over vertex-local objects. A transaction acquires read locks for vertices it inspects and write locks for vertices it modifies; locks are released only after commit or abort. The existing no-wait retry policy can be used to avoid deadlock: if a transaction fails to acquire a required lock, it releases the locks acquired in the current attempt, backs off, and retries or aborts.

This extension is mainly suitable for short OLTP-style graph transactions with small read and write sets. It is not the default mode for long-running graph analytics, because analytical traversals may touch a large fraction of the graph and would hold many read locks for a long time, substantially increasing reader-writer interference and reducing the benefit of AVBGraph's snapshot-oriented design. Therefore, our evaluation focuses on the snapshot-isolated dual-grained and coarse-grained modes.

E DETAILED DERIVATION OF THE COST MODEL

This appendix provides the derivation details omitted from Section 6 and explains the refinements used in the AVB cost formula.

E.1 Expected Number of Affected Edges and Candidate VBs

Consider a snapshot scan of a vertex v with out-degree d . We assume that updates to each edge follow an independent Poisson process with mean μ updates per edge per epoch. Over a lag window of Δ epochs, the number of updates received by one edge is therefore

$$X_e \sim \text{Poisson}(\mu\Delta).$$

An edge is affected during the lag window if it is updated at least once. Thus,

$$\Pr[X_e \geq 1] = 1 - \Pr[X_e = 0] = 1 - e^{-\mu\Delta}.$$

By linearity of expectation, the expected number of distinct affected edges among the d outgoing edges of v is

$$d_h = \sum_{e \in N(v)} \Pr[X_e \geq 1] = d(1 - e^{-\mu\Delta}).$$

At the VB level, we ask whether vertex v contributes any historical record to a given epoch. Since the aggregate update rate over the d outgoing edges is $d\mu$ per epoch, the number of updates to v in one epoch follows

$$Y_v \sim \text{Poisson}(d\mu).$$

Therefore, the probability that v has a non-empty VB in one epoch is

$$\Pr[Y_v \geq 1] = 1 - e^{-d\mu}.$$

Over Δ lagging epochs, the expected number of non-empty candidate VBs that a snapshot scan may need to inspect is

$$B_v(\Delta) = \Delta(1 - e^{-d\mu}).$$

E.2 N2O Chain Cost

In an N2O-style layout, visibility is resolved independently for each edge. After scanning the current adjacency, a reader may need to follow version-chain pointers to skip versions that are newer than its snapshot. For one edge, the expected number of such newer historical versions in the lag window is $\mu\Delta$. Across d edges, the expected number of chain probes is $d\mu\Delta$.

Thus, the expected chain-based visibility cost is

$$C_{\text{chain}}(v, \Delta) \approx dc_s + d\mu\Delta c_r.$$

Here, dc_s is the cost of scanning the current adjacency, and $d\mu\Delta c_r$ is the dominant random or pointer-chasing cost of resolving historical visibility through edge-level chains.

E.3 AVB Cost and Its Refinements

In AVB, historical versions are grouped into VBs. A snapshot scan still pays dc_s to scan the current adjacency, but historical visibility is resolved at block granularity. For each non-empty candidate VB, the reader pays one random block-entry cost, giving the term

$$B_v(\Delta)c_r.$$

After entering candidate VBs, the remaining historical records are scanned sequentially. A basic approximation would charge one sequential access for each useful historical record:

$$C_{\text{AVB}}^{\text{basic}}(v, \Delta) \approx dc_s + B_v(\Delta)c_r + d_h c_s.$$

The refined formula uses three accounting rules.

First, AVB pays random access at VB granularity rather than at individual edge-version granularity. Thus, the random term is $B_v(\Delta)c_r$ instead of $d\mu\Delta c_r$.

Second, the useful historical work is bounded by the number of distinct affected edges, d_h , because multiple updates to the same edge within the lag window still contribute at most one visible replacement record to a snapshot scan.

Third, block-entry accounting avoids charging the first useful touch in a candidate VB twice. If a candidate VB is pure, meaning that all records in the block are visible to the snapshot, the block-entry access already accounts for one useful touch. For a non-pure VB, the scan may additionally touch one invisible boundary record, which offsets this saving. Hence only pure VBs receive a one-touch credit. Moreover, the remaining sequential work cannot exceed the total number of historical records beyond the first record of each candidate VB, namely $d\mu\Delta - B_v(\Delta)$.

Therefore, the effective sequential work is

$$x_{\text{seq}}(\Delta) = \min \{d_h - \min\{q_p B_v(\Delta), d_h\}, d\mu\Delta - B_v(\Delta)\}.$$

E.4 Expected Speedup

Combining the two costs, the expected speedup of AVB over N2O-style chain-based visibility resolution is

$$S(\Delta) = \frac{C_{\text{chain}}(v, \Delta)}{C_{\text{AVB}}(v, \Delta)} \approx \frac{dc_s + d\mu\Delta c_r}{dc_s + B_v(\Delta)c_r + x_{\text{seq}}(\Delta)c_s}.$$

This expression captures the main trend: N2O pays visibility-resolution cost through many edge-level random probes, while AVB replaces most of these probes with block-entry checks and sequential in-block scans. The benefit therefore increases when the lag window Δ , the degree d , the random-access penalty c_r/c_s , or the pure-block fraction q_p increases.

F COMPONENT STUDY AND BASELINE DETAILS

This appendix supplements the evaluation with (i) a component study isolating two epoch-management design choices, and (ii) the detailed modifications applied to RapidStore. All component-study experiments use the same concurrent setting as Section 7.3 (32 writer and 32 reader threads on Graph500-24, readers running PageRank), so each mechanism is measured under realistic read-write interference rather than in isolation. Results are shown in Figure 14.

F.1 Effect of Epoch Size N

We vary the [target expected epoch size](#) N from 10^3 to 10^7 . Figure 14(a) shows that ingestion throughput [generally rises with \$N\$ and approaches a plateau at large \$N\$](#) : it grows from 1.78 to 5.49 MEPS for AVBGraph, and from 1.73 to 4.76 MEPS for AVBGraph-dual. A larger epoch lets more records share the same invalidation [epoch](#),

producing larger Version Blocks that amortize visibility checks, publication, and reclamation over more records and that are scanned more sequentially.

PageRank latency (Figure 14(b)) stays essentially flat across N (roughly 4–6 s, within run-to-run noise). This is expected: PageRank is a long scan-heavy analytic whose runtime dominates the epoch publication delay, so enlarging N does not slow it. The main cost of a larger N is less frequent publication in the coarse-grained mode: under coarse-grained visibility an epoch is the unit of reader-visible publication, so increasing N increases the expected number of updates between publication boundaries; it does not make updates visible in fixed groups of exactly N . This freshness coarsening, rather than any latency penalty, is what the dual-grained extension (Section 4.3) is designed to refine. Thus N trades batching efficiency against visibility granularity for analytics-dominated workloads, while the dual-grained mode decouples the two when finer freshness is required.

F.2 Effect of Sampled Sealing

We replace sampled sealing with an exact per-update counter that every update increments to detect the sealing threshold (the “x” markers at $N = 10^7$ in Figure 14). Removing sampled sealing reduces ingestion throughput from 5.49 to 4.87 MEPS for AVBGraph (−11.3%) and from 4.76 to 3.88 MEPS for AVBGraph-dual (−18.5%), while PageRank latency is essentially unchanged. Under 32 concurrent writers an exact global counter becomes a contention point on the common write path, whereas sampled sealing touches shared state only on a sampled hit and therefore keeps the write path lightweight. The larger drop for AVBGraph-dual is consistent with its write path already holding vertex-local locks until commit, so additional global contention compounds more sharply. Overall, sampled sealing preserves higher update throughput at comparable PageRank latency in this setting.

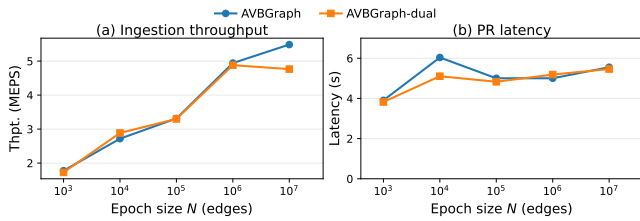


Figure 14: Component study under 32W+32R on Graph500-24. Throughput rises with N and saturates; PageRank latency shows no monotonic trend across the tested values of N . The “x” markers replace sampled sealing with an exact per-update counter at $N = 10^7$.

F.3 RapidStore Modifications

RapidStore’s public implementation is tuned for its own concurrent-query benchmark and exposes two issues outside that setting, both of which we had to address to run it under our mixed read–write workloads.

The first modification corrects synchronization of the reader-trace metadata. Without this synchronization, concurrent snapshot registration and release can leave the metadata in an inconsistent

state and cause incorrect or crashing executions. We protect these operations so that the concurrent experiments execute correctly. This change is required for correctness and introduces additional coordination on the read path.

The second modification applies to the property-update paths. We retain RapidStore’s ART update path but bypass the copy-on-write and associated resource-maintenance work of RangePropertyVec and RangeTree, which is unstable under our update workloads. Bypassing these paths removes maintenance work and relaxes their original correctness requirements, thereby biasing update performance upward.

The two modifications have opposing performance effects: the correctness synchronization adds coordination, whereas bypassing maintenance removes work. We therefore report this variant as *RapidStore**, a correctness-relaxed reference with an optimistic bias on these update paths, rather than as a strict performance upper bound. We omit its memory footprint because the bypassed maintenance makes it incomparable with systems executing their complete update paths.

G ADDITIONAL UPDATE EVALUATION

G.1 General-Update Workload Generation

We provide additional generation and execution details for the ID, IDM, and M100 workloads in Section 7.1. Before execution, the benchmark generates the complete update sequence as five rounds, totaling $5|E|$ single-edge update transactions. Within each round, edge indices from the input graph are randomly shuffled and partitioned into the update types required by the workload. All systems are given the same pre-generated operation sequence. Targets are sampled without replacement within a round, while reshuffling in each subsequent round allows the same edge to be updated again.

For a deletion–insertion pair, an existing edge is first deleted and then reinserted with its original source, destination, and value. The two updates are executed consecutively by the same writer but remain separate single-edge transactions. This construction ensures a valid insertion target and restores the graph size after each pair. Modification targets are disjoint from deletion–insertion targets within the same round and are assigned a newly generated edge value. Thus, ID and IDM remain graph-size stable while allowing repeated updates to the same edges across rounds. Each insertion, deletion, or modification is executed as a single-edge transaction.

The benchmark adds no application-level retry loop. Any retry or abort handling internal to a system is included in the measured wall-clock time. Throughput is reported as completed update operations per second, excluding initial graph loading and workload generation.

G.2 Write Scalability

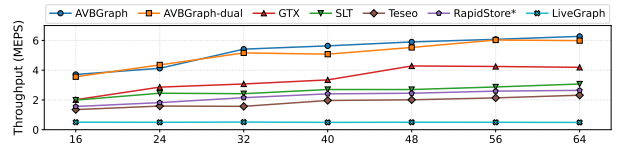


Figure 15: Edge insertion throughput on Graph500-24 with 16–64 writer threads.

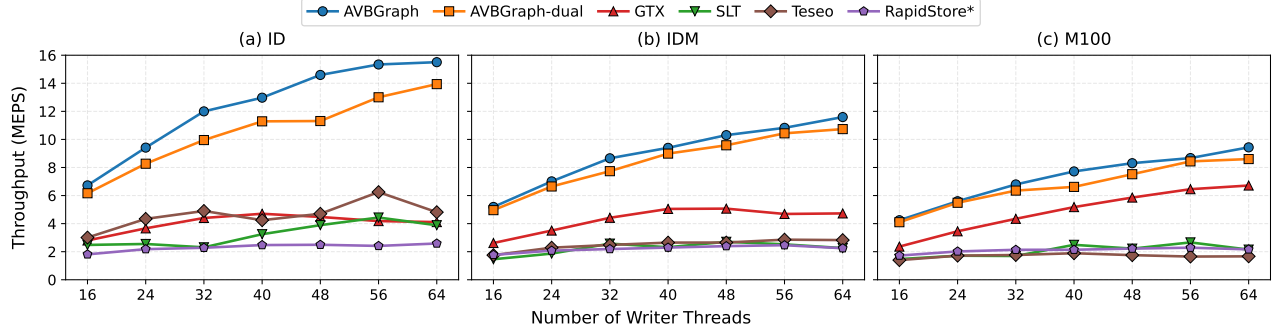


Figure 16: ID, IDM, and M100 throughput on Graph500-24 with 16–64 writer threads.

We further evaluate insertion throughput on Graph500-24 while varying the number of writer threads from 16 to 64. As shown in Figure 15, both AVBGraph modes remain above the baselines across the tested range. Their throughput increases with writer concurrency and gradually approaches saturation at higher thread counts, supporting the 64-writer setting used in Section 7.1.

We also evaluate ID, IDM, and M100 on Graph500-24 with 16–64 writer threads, using the workload generation described in Appendix G.1. Figure 16 shows that both AVBGraph modes achieve higher throughput than the baselines at every tested thread count under all three workloads. Their throughput continues to increase toward 64 threads, although the gains become more gradual at higher concurrency. These results show that AVBGraph’s write scalability extends beyond insertion-only workloads to mixed insertions and deletions and edge-value modifications. As in Section 7.1, LiveGraph exceeds the time limit, so no throughput results are reported for it.

G.3 Concurrent Mixed Updates under HotSet

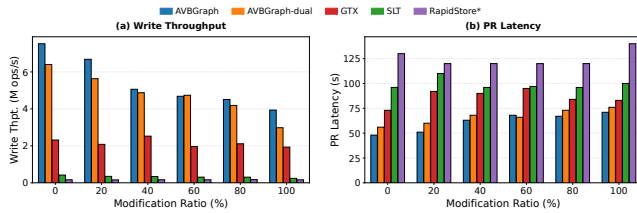


Figure 17: Concurrent mixed updates under HotSet on Graph500-24 with eight PageRank tasks. The modification ratio varies from 0% to 100%, with the remaining operations split equally between insertions and deletions: (a) write throughput and (b) PageRank latency.

The concurrent workloads in Section 7.3 use only value modifications to existing edges, i.e., 0% insertions, 0% deletions, and 100% modifications. To evaluate whether the observed behavior extends to other update compositions, we further vary the mixture of edge insertions, deletions, and value modifications under HotSet.

Workload generation. We use Graph500-24 with 32 writer and 32 reader threads and eight concurrent PageRank tasks. The HotSet consists of outgoing edges of a degree-weighted sample of 0.1% of the vertices. We vary the modification ratio from 0% to 100%; the remaining operations are split equally between insertions and

deletions. Each operation is a single-edge transaction, and each configuration executes approximately $5|E|$ updates using the same workload generation across systems.

Figure 17 reports write throughput and PageRank latency. As the modification ratio increases, the write throughput of both AVBGraph variants generally decreases. A larger modification ratio causes a larger fraction of operations to repeatedly modify the values of existing hotspot edges. Nevertheless, AVBGraph and AVBGraph-dual maintain substantially higher write throughput than the baselines throughout the sweep.

GTX also operates at a considerably lower write rate than the two AVBGraph variants and shows no corresponding latency advantage as the workload becomes more modification-heavy. SLT sustains still lower write throughput, while its PageRank latency remains substantially higher than that of AVBGraph. RapidStore* shows relatively little variation in write throughput, but its absolute update rate remains much lower; its PageRank latency is also higher under this eight-task workload. At every tested modification ratio, AVBGraph achieves the highest write throughput and lowest PageRank latency among the compared systems.

The PageRank latency of both AVBGraph variants increases only moderately as the modification ratio rises. Thus, their higher update throughput does not come with a corresponding sharp increase in analytical latency, even as a larger fraction of the workload repeatedly modifies existing hotspot edges.

Overall, these results complement the modification-only concurrent experiments in Section 7.3. AVBGraph’s advantage persists as the workload changes from balanced deletion–insertion operations to increasingly modification-heavy execution. This behavior is consistent with AVBGraph’s block-oriented historical organization: displaced versions from multiple edges are grouped into Version Blocks, allowing readers to prune irrelevant history at block granularity and sequentially scan the remaining records rather than repeatedly traversing independent per-edge version chains.